Chapter 3
Hash Functions

Stefan Dziembowski
www.crypto.edu.pl/Dziembowski

University of Warsaw

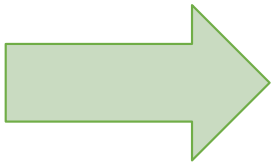


Secure communication

	encryption	**authentication**
private key	1 private key encryption	2 private key authentication
public key	3 public key encryption	4 signatures

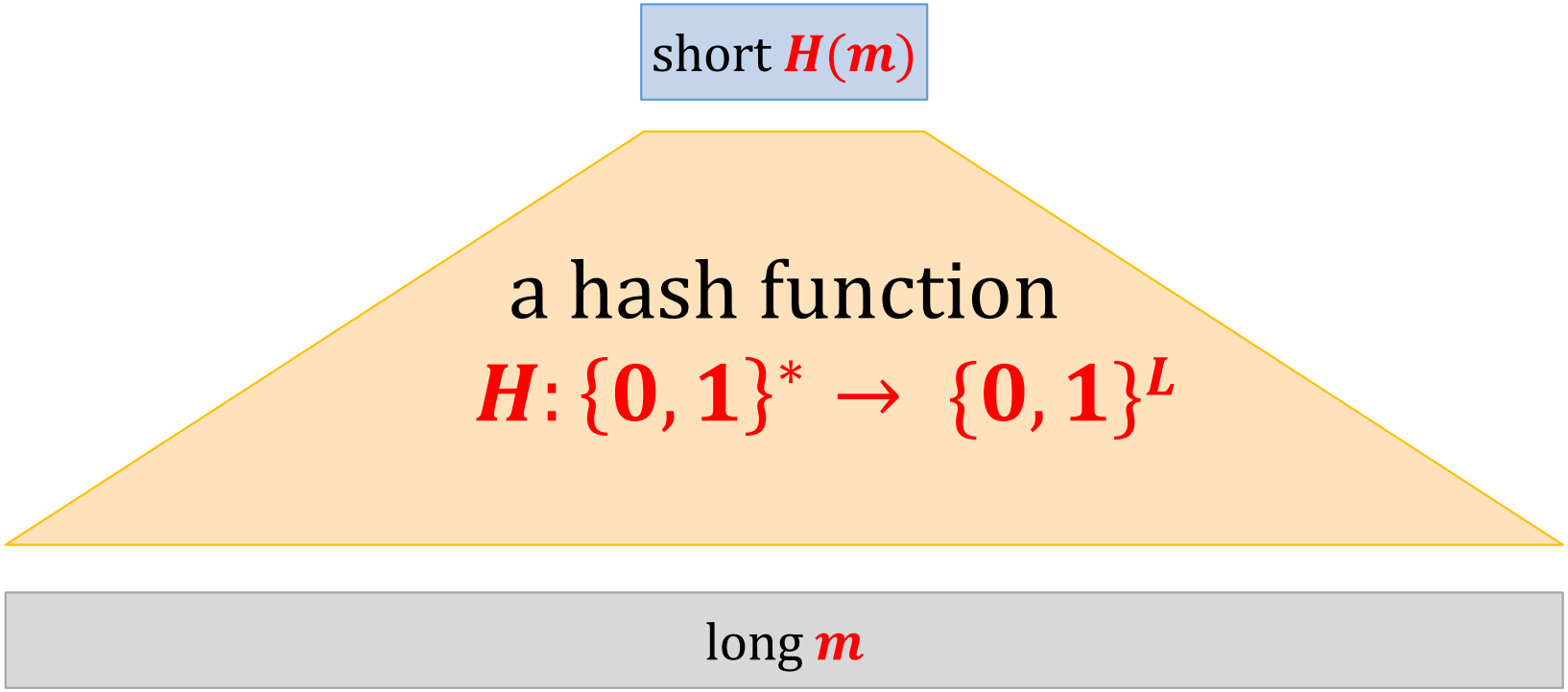
we will discus this on the next lecture, but first, we talk about the hash functions

Plan

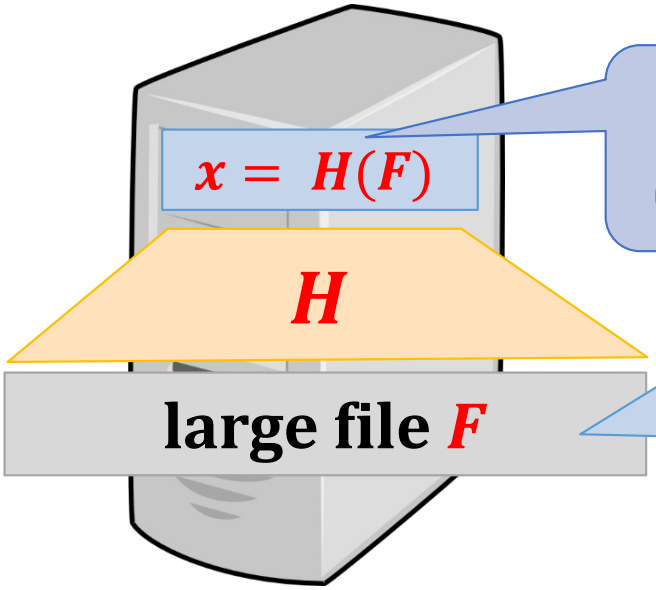


1. Introduction and definitions
2. Hash function design paradigms
3. Other uses of hash functions
4. Real-life constructions

Hash functions



Example of an application: file fingerprinting



"**fingerprint**"
(publish this through a **secure channel**)

everybody who downloads F can verify its correctness by computing $F(H)$ and comparing with x.

Example (ubuntu.com):

64-bit PC (amd64, x86_64) (Recommended)

1. Ubuntu 15.10 "Wily Werewolf" 40MB (MD5: d1498749e073ef7fd09f84478f299bea, SHA1: aa659499dc300fe2be00d756180793093cc15014)
2. Ubuntu 15.04 "Vivid Vervet" 40MB (MD5: 3b00a4573b11fb1f85eaa05918971789, SHA1: 97282a3b066de4ee4c9409979737f3911f95ceab)

What properties should a hash function H have?

Minimal requirement: second preimage-resistance.

More precisely, the following problem should be **hard for any efficient adversary A**:

- **given**: "random" $m \in \{0,1\}^*$
- **find**: $m' \neq m$ such that

$$H(m) = H(m')$$

Q: Is it enough?

Second preimage resistance may be in many cases be too weak

What if the adversary can somehow influence the choice of m?

For example: Ubuntu has many contributors. What if one of them is malicious?

Idea: modify the definition by allowing the adversary to choose m himself.

New game for the adversary:

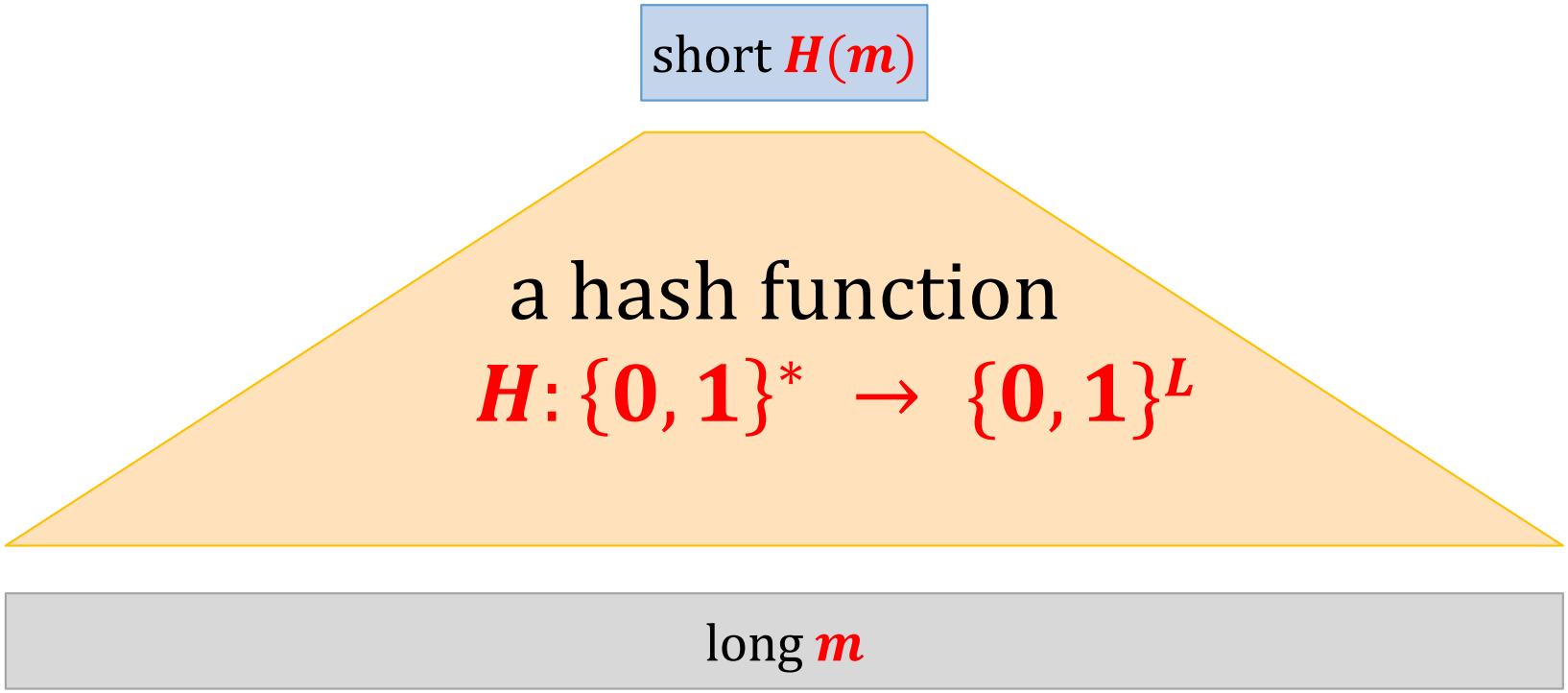
if this problem is hard then a function is called "**collision-resistant**"

find: m
find: $m' \neq m$ such that $H(m) = H(m')$

$\Leftrightarrow$

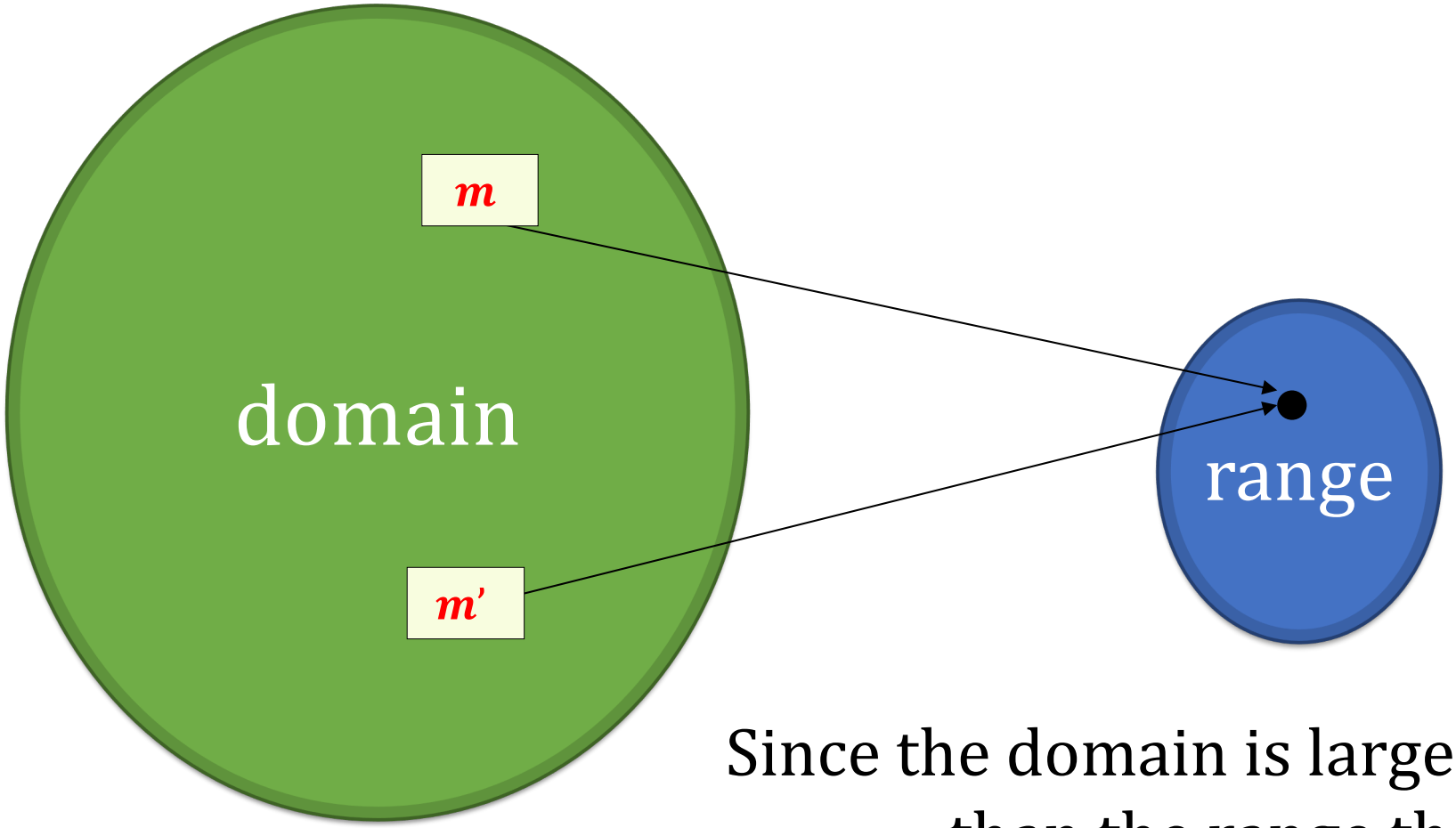
find: $m \neq m'$ such that $H(m) = H(m')$

Collision-resistant hash functions



Requirement: it should be hard to find a pair (m, m') such that $m \neq m'$ and $H(m) = H(m')$

Collisions always exist



Since the domain is larger than the range the collisions must exist.

“Practical definition”

H is a **collision-resistant hash function** if it is “*practically impossible to find collisions in* ***H***”.

Popular hash functions (we will present them in more detail on the next lecture):

- **MD5** (broken)
- **SHA1** (broken)
- **SHA256**

} based on **the Merkle-Damgård transformation**

- **Keccak (SHA3)** ⇦ based on the **sponge construction**

Hash functions can also be constructed using mathematical tools like number theory.

How to formally define “collision resistance”?

Idea: Say something like: H is a **collision-resistant hash function** if

$\forall$ efficient adversary A $\quad P(A \text{ finds a collision in } H)$ is small

Problem

For a fixed H there **always exist** a constant-time algorithm that “finds a collision in H” in **constant time**.

It may be hard to **find** such an algorithm, but it always exists!

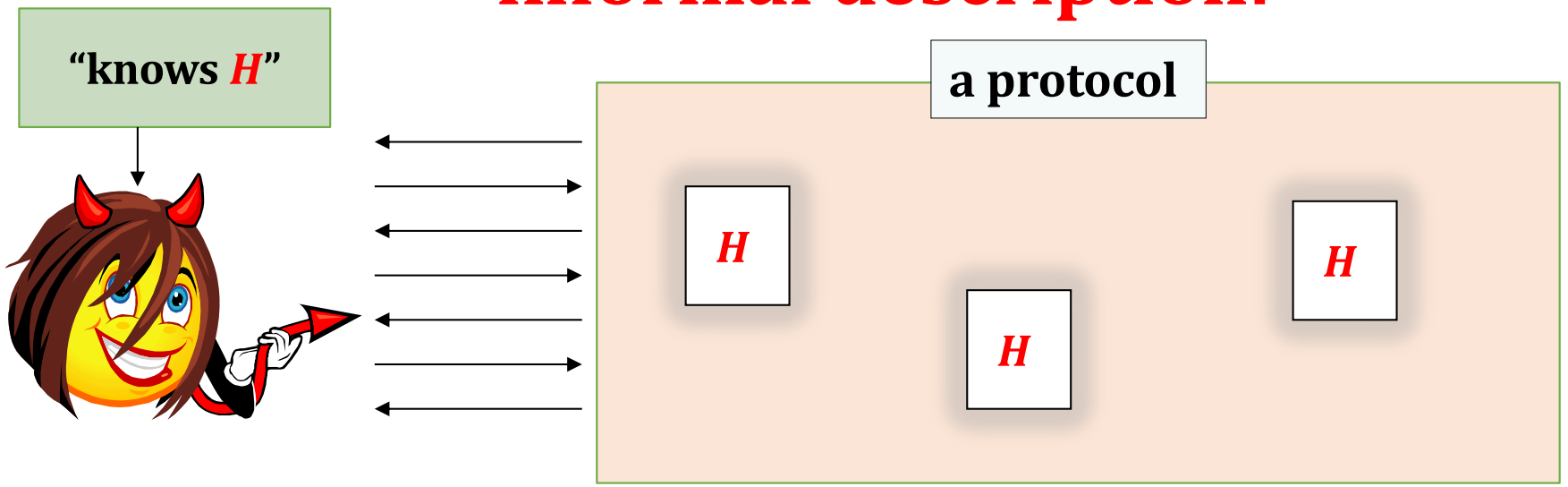
Solution

When we prove theorems we will always consider

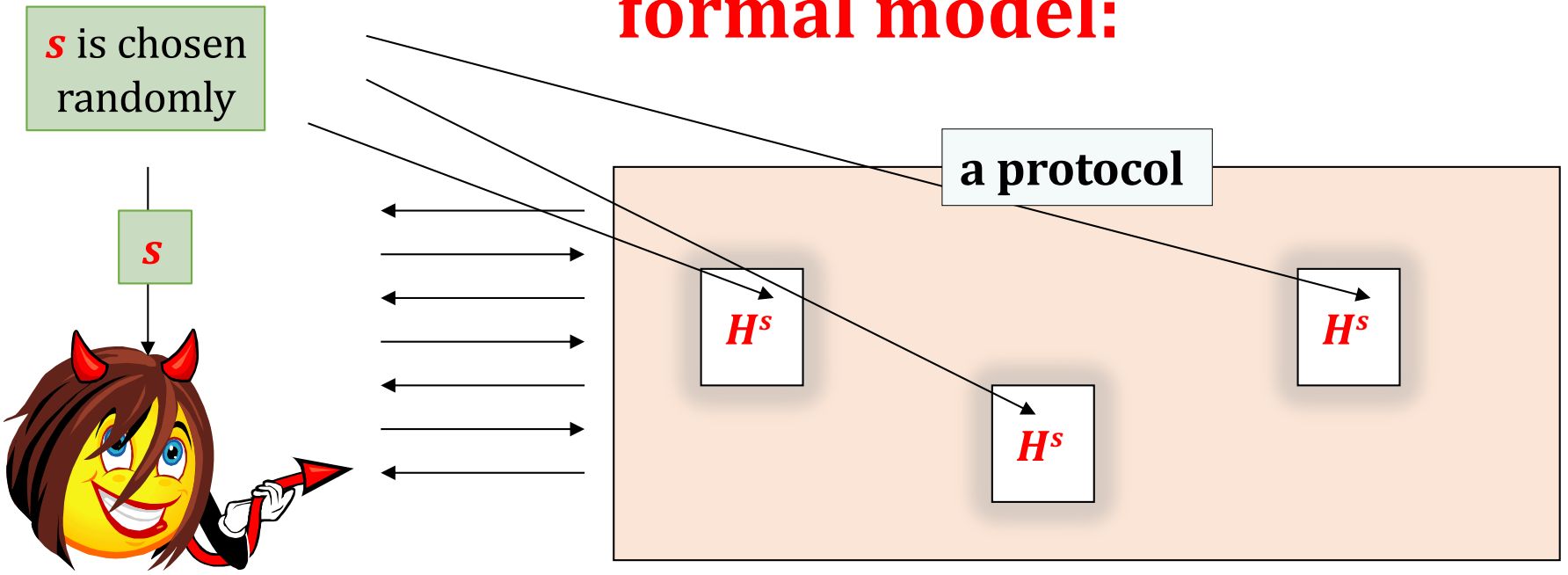
families of hash functions indexed by a key s:

$$\{H^s\}_{s \in \text{keys}}$$

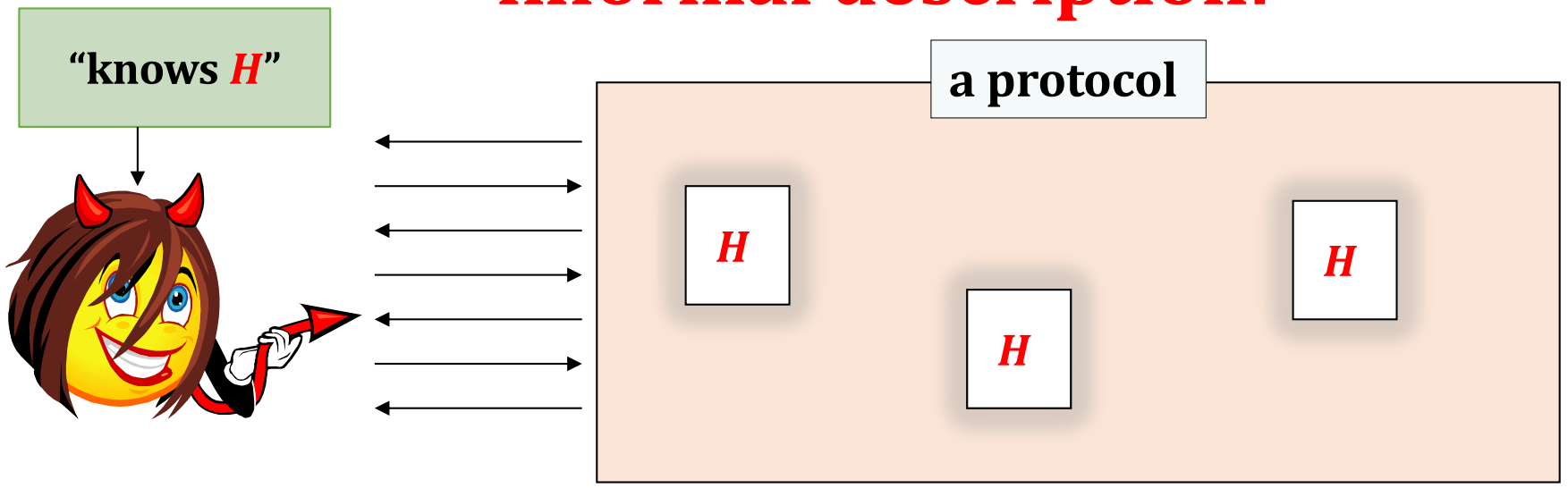
informal description:



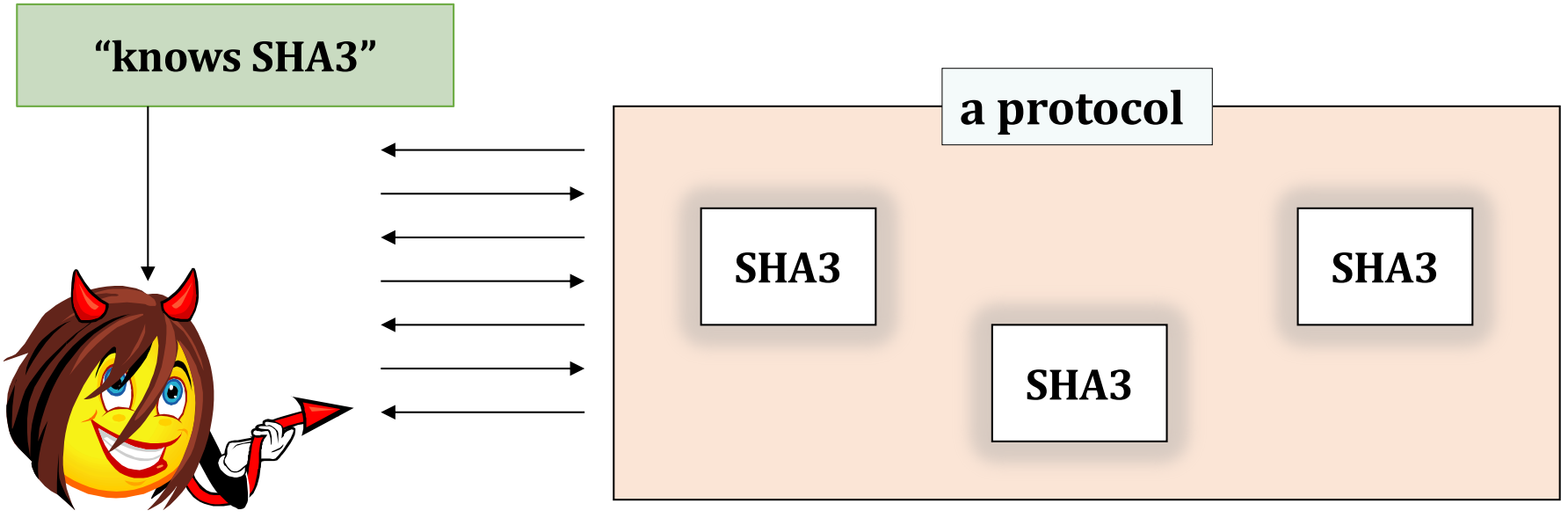
formal model:



informal description:



real-life implementation (example):



Hash functions – the functional definition

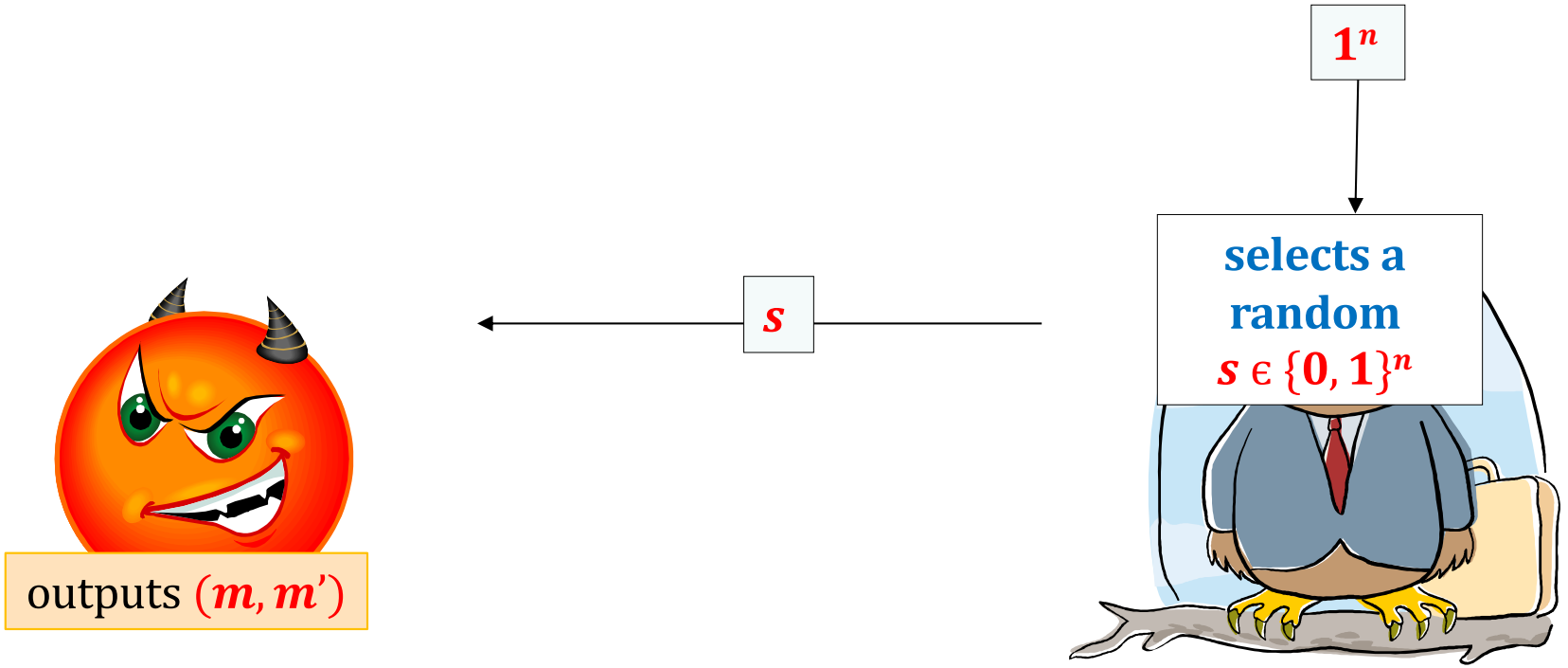
A **hash function** is a probabilistic polynomial-time algorithm H such that:

H takes as input a key $s \in \{0,1\}^n$ and a message $m \in \{0,1\}^*$ and outputs a string

$$H^s(m) \in \{0,1\}^{L(n)}$$

where $L(n)$ is some fixed function.

Hash functions – the security definition [1/2]



We say that adversary A breaks the function H if $m \neq m$ and $H^s(m) = H^s(m')$.

Hash functions – the security definition [2/2]

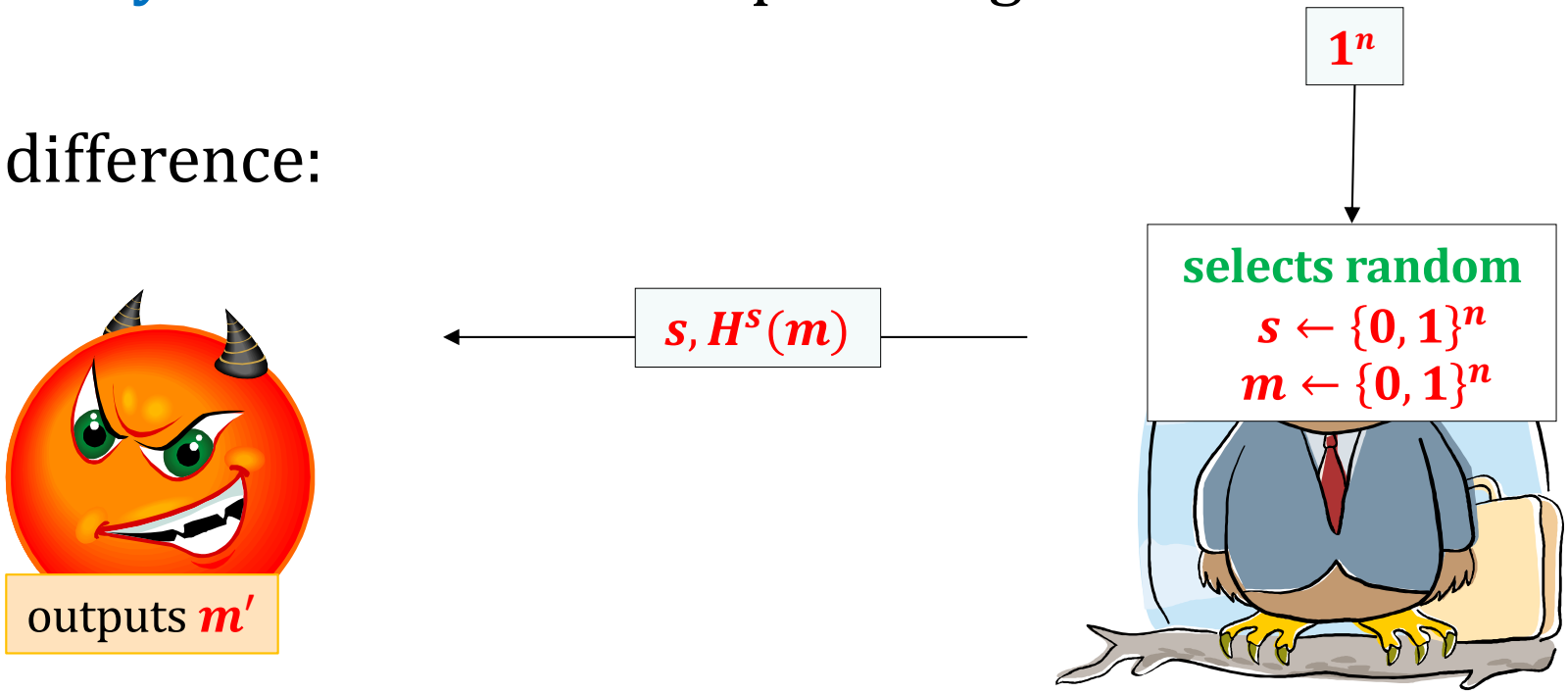
H is a **collision-resistant hash function** if

$\forall$ polynomial-time adversary A $\quad P(A \text{ breaks } H)$ is negligible

A weaker requirement: **pre-image resistance**

Intuitively: "it's hard to find a pre-image of H^s"

Main difference:

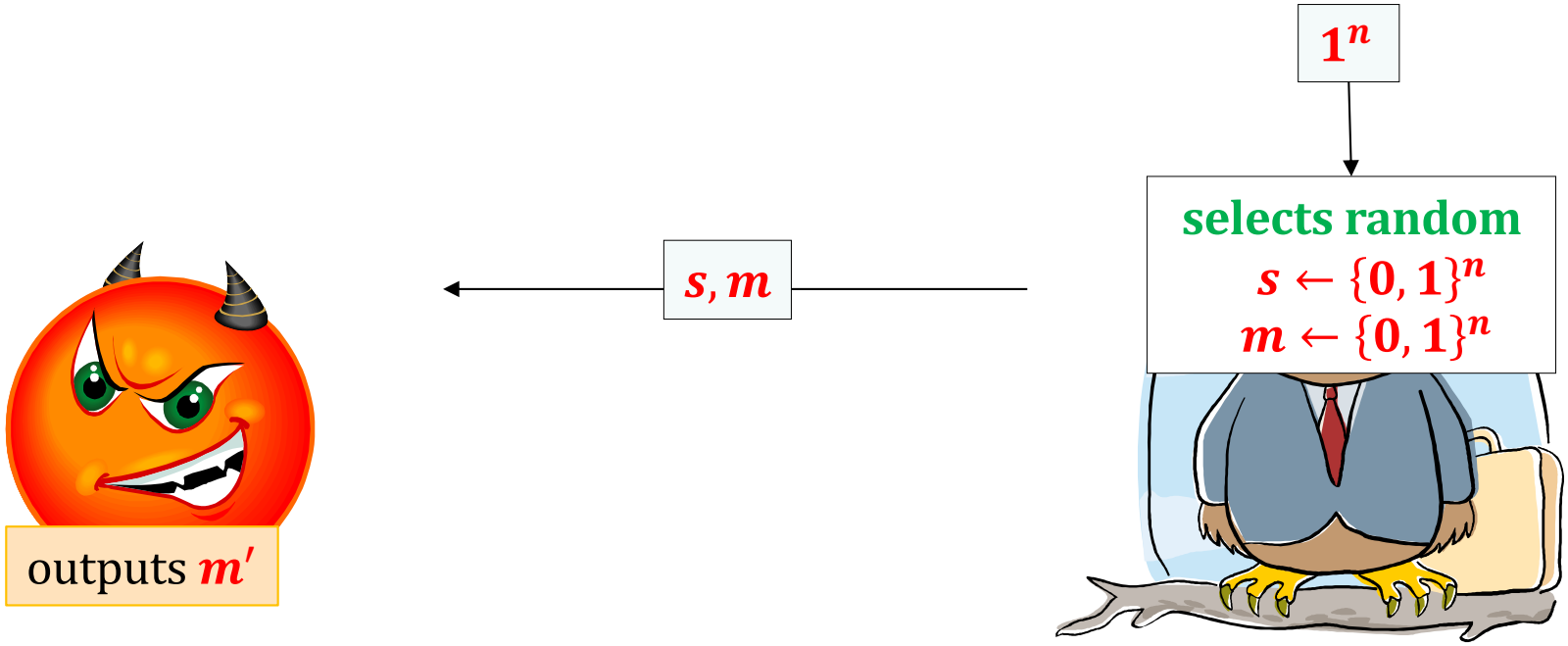


We say that adversary A breaks the function H if

$$H^s(m) = H^s(m')$$

Yet another requirement: **second pre-image resistance**

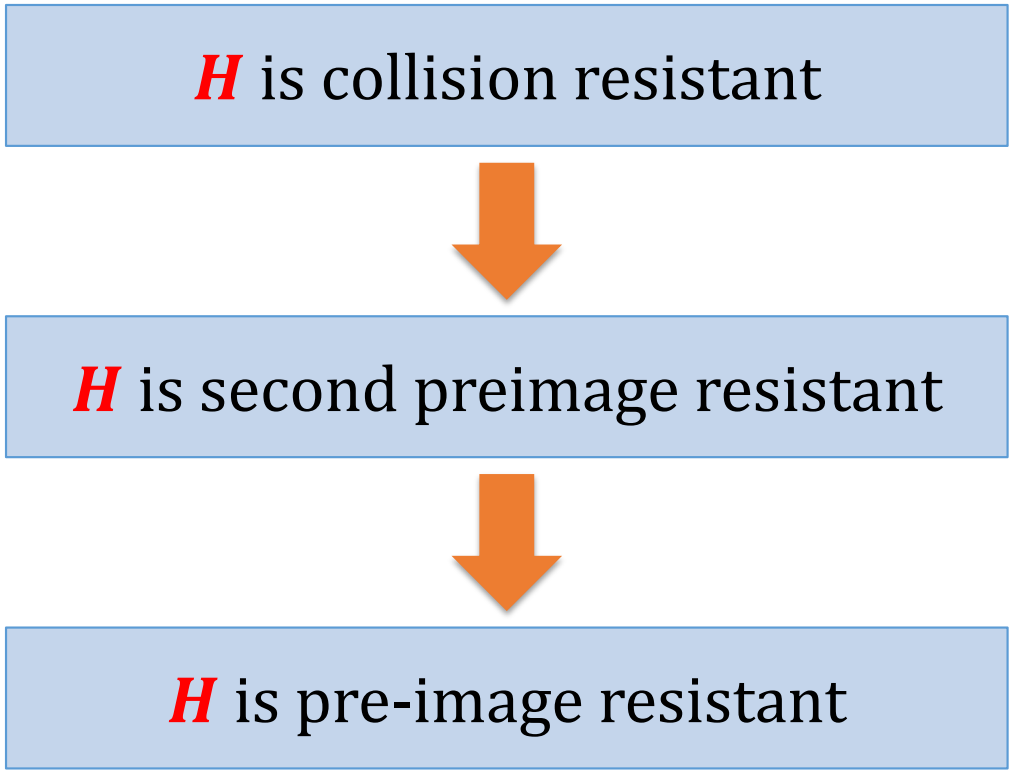
Intuitively: "it's hard to find a second pre-image of H^s"



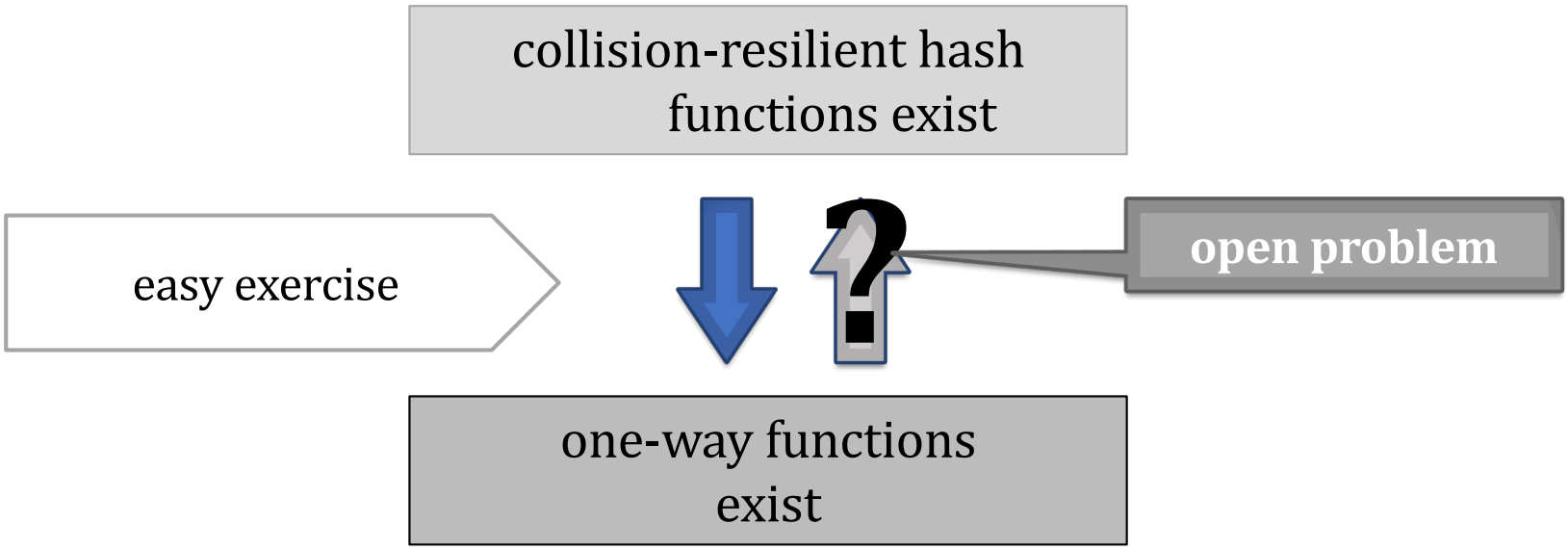
We say that adversary A breaks the function H if $m' \neq m$ and $H^s(m) = H^s(m')$

Fact

The following implications hold:



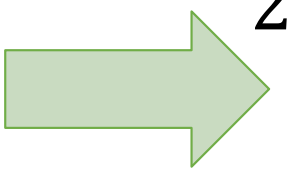
Do collision-resilient hash functions belong to minicrypt?



[D. Simon: Finding Collisions on a One-Way Street: Can Secure Hash Functions Be Based on General Assumptions? 1998]:

there is no “black-box reduction”.

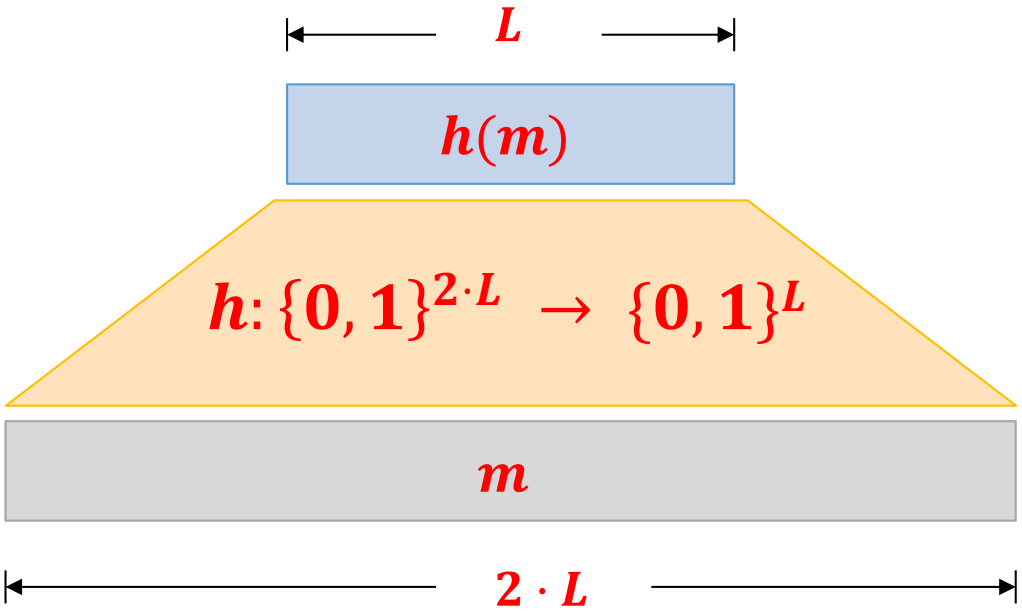
Plan



1. Introduction and definitions
2. Hash function design paradigms
    1. Merkle-Damgård transform
    2. Sponge construction
3. Other uses of hash functions
4. Real-life constructions

A common method for constructing hash functions

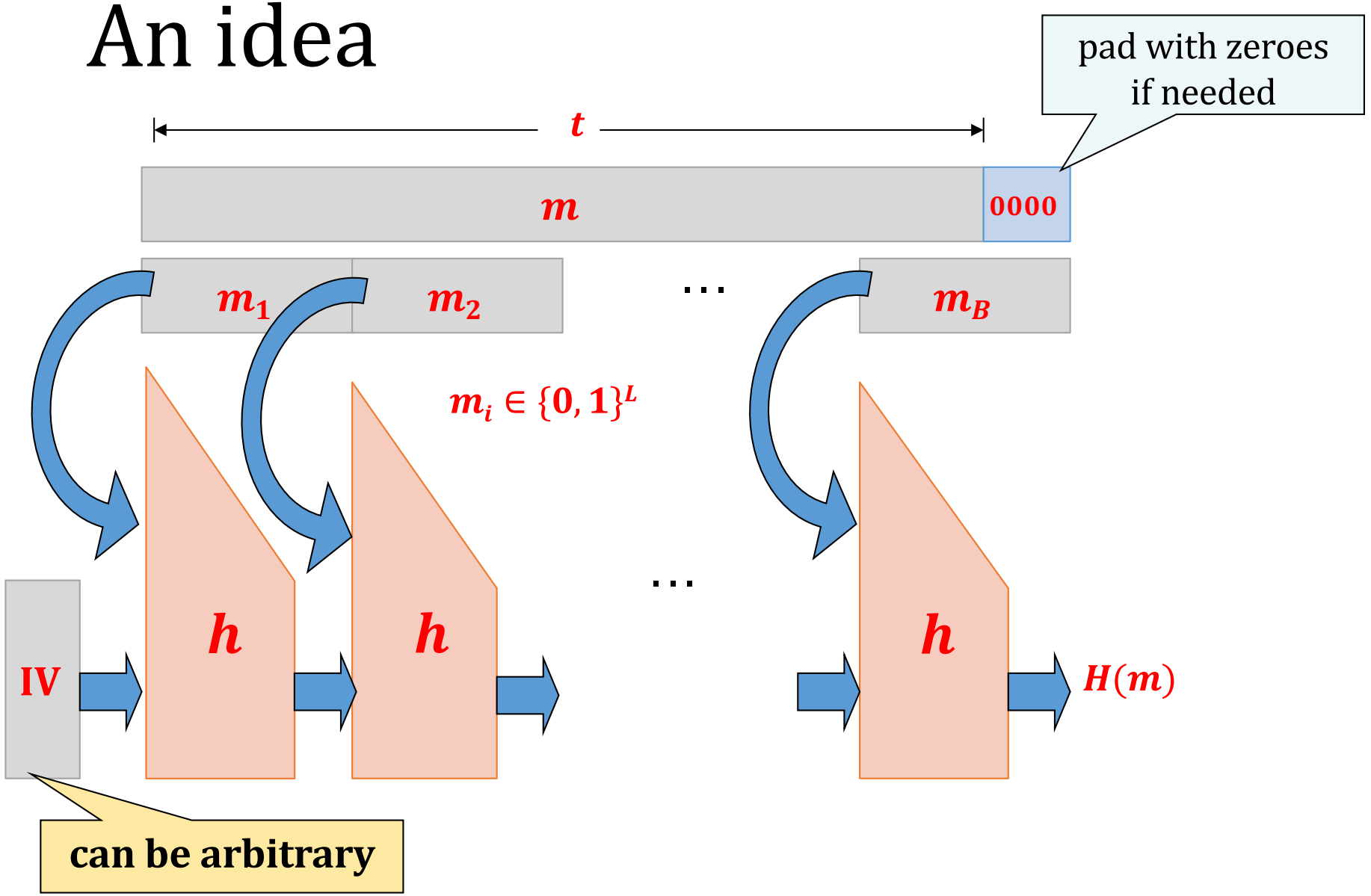
1. Construct a "***fixed-input-length***" collision-resistant hash function



   Call it: a collision-resistant **compression function**.

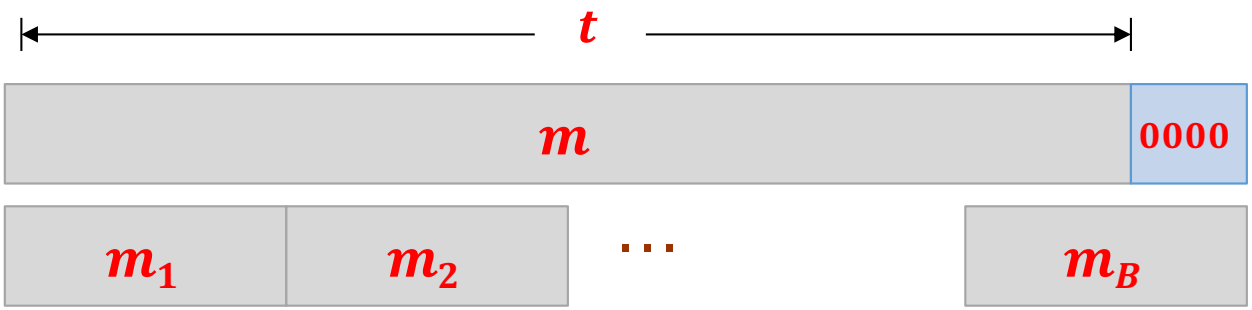
2. Use it to construct a hash function.

An idea



This doesn't work...

Why is it wrong?



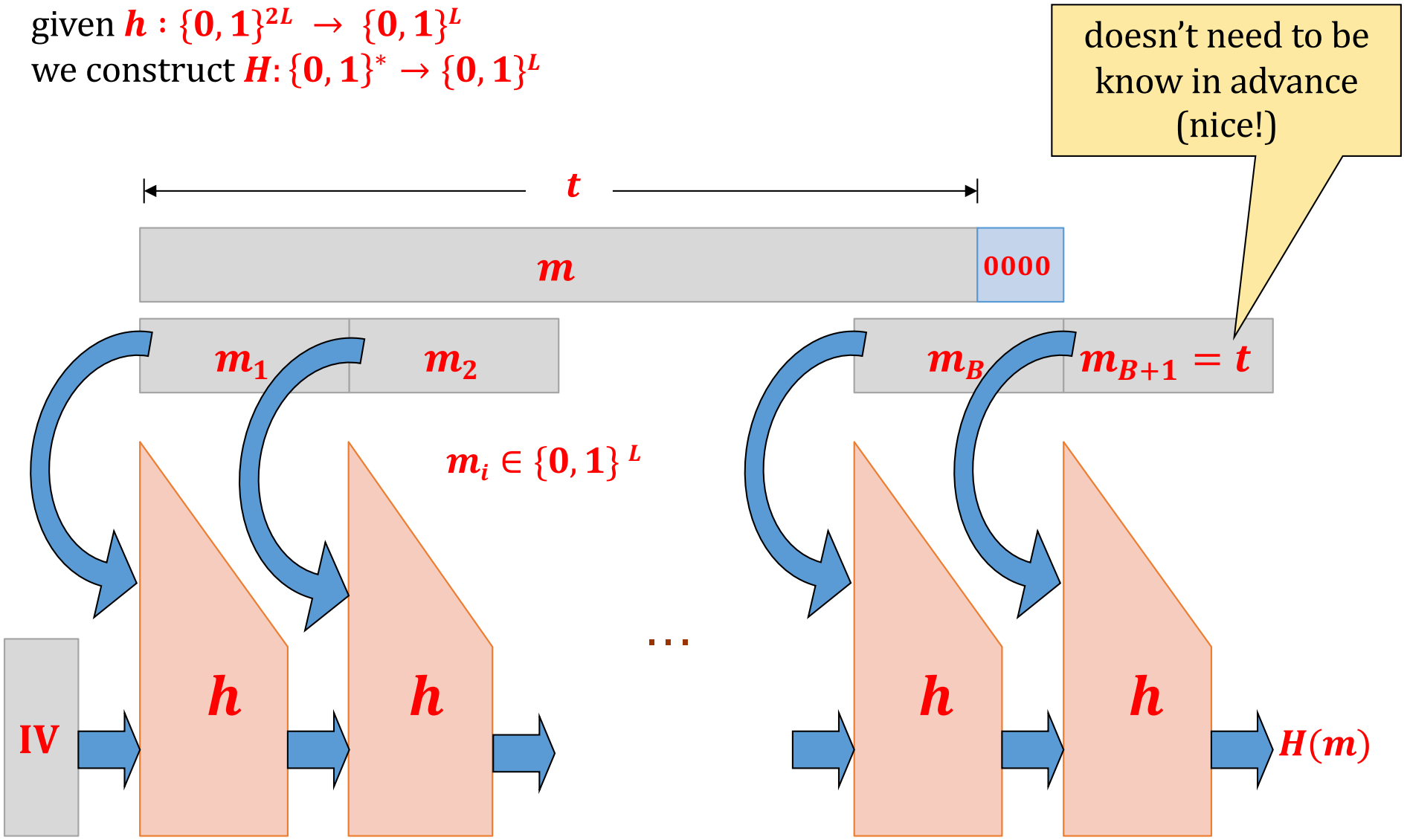
If we set $m' = m \,||\, 0000$ then $H(m') = H(m)$.

Solution: use some **prefix-free encoding**, e.g., add a block encoding "t".



Merkle-Damgård transform

given $h : \{0,1\}^{2L} \rightarrow \{0,1\}^{L}$
we construct $H: \{0,1\}^{*} \rightarrow \{0,1\}^{L}$



This construction is secure

We would like to prove the following:

Theorem

If

$$h: \{0,1\}^{2L} \rightarrow \{0,1\}^{L}$$

is a collision-resistant **compression** function

then

$$H: \{0,1\}^{*} \rightarrow \{0,1\}^{L}$$

is a collision-resistant **hash** function.

But wait….
It doesn't make sense…

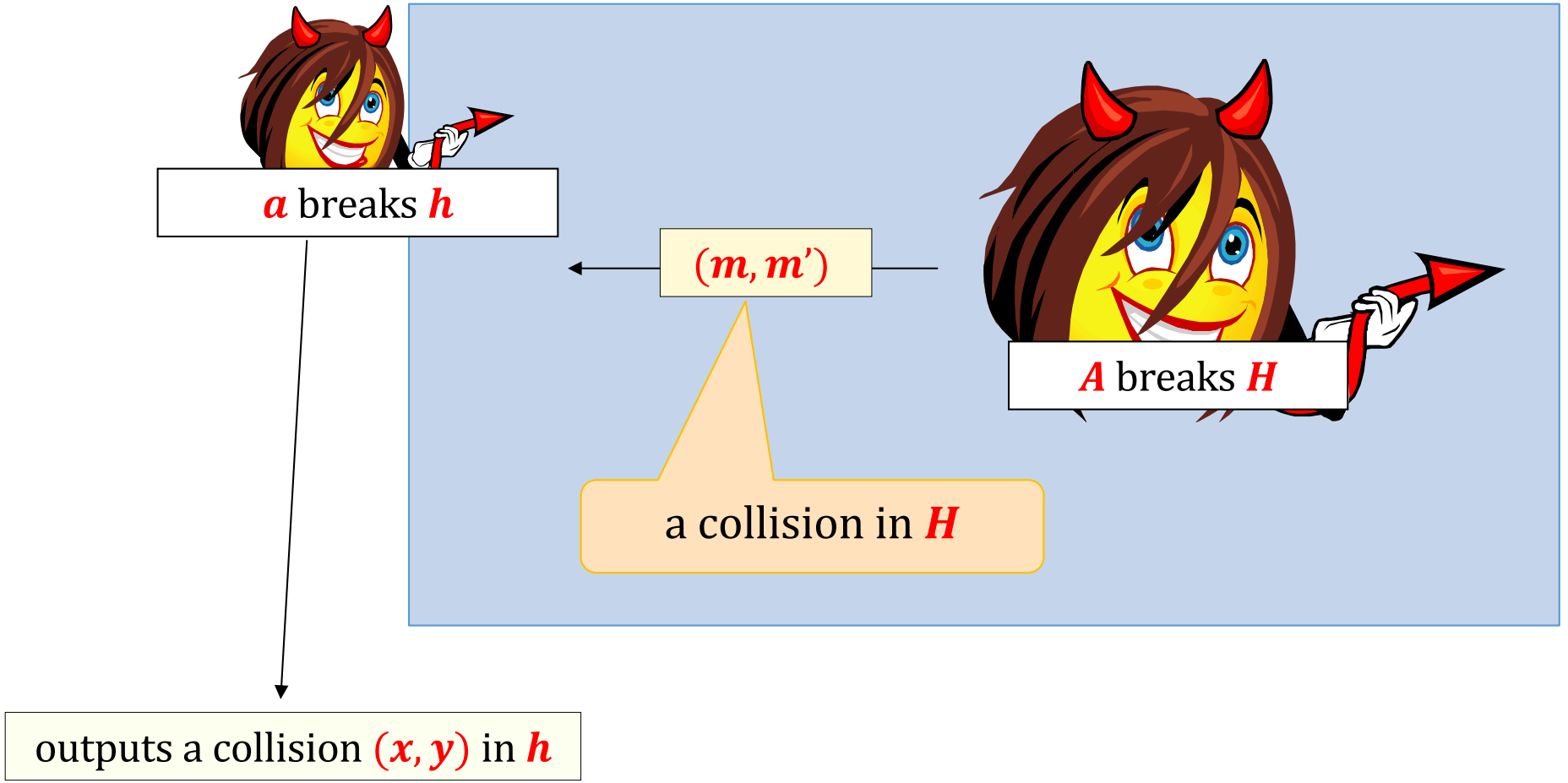
What to do?

To be formal, we would need to consider

families of functions
h and ***H***
indexed by key ***s***

Let's stay on the **informal level** and argue that:

"if one can find a collision in ***H*** then one can find a collision in ***h***"


a breaks h
(m, m')
A breaks H
a collision in H
outputs a collision (x, y) in h

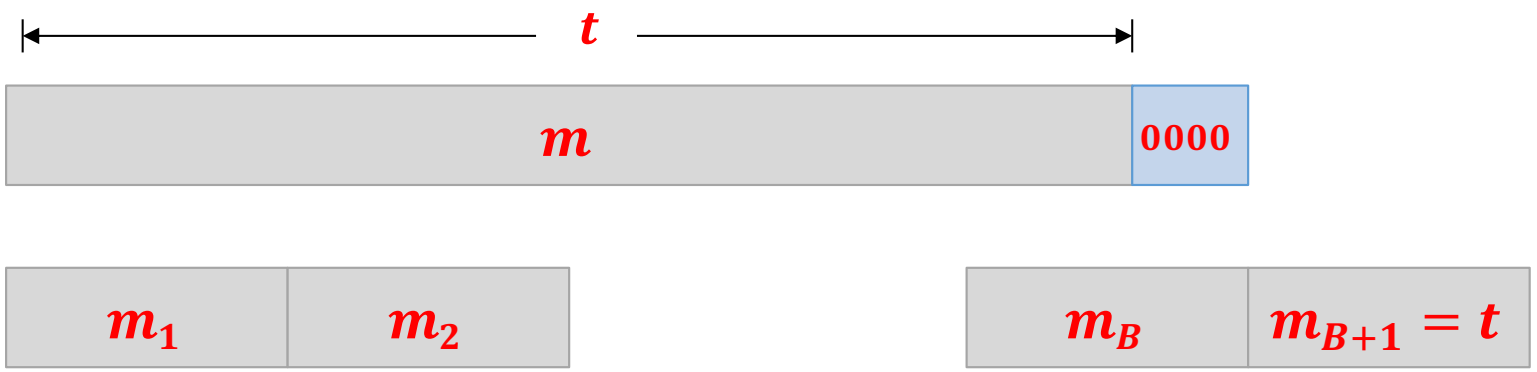
How to compute a collision (x, y) in h from a collision (m, m') in H?

We consider two cases:

1. $|m| = |m'|$

2. $|m| \neq |m'|$

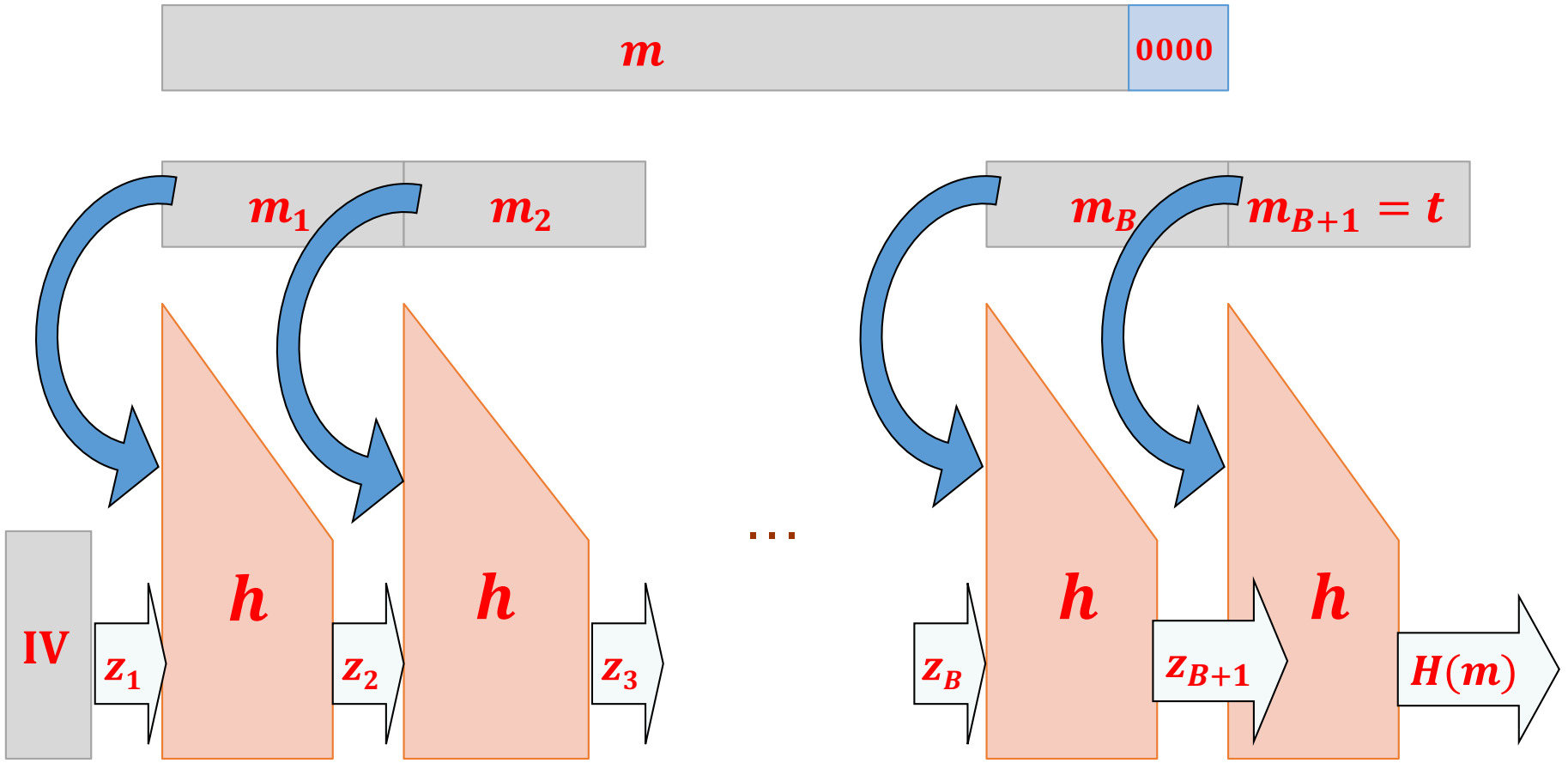
Case 1: $|m| = |m'|$





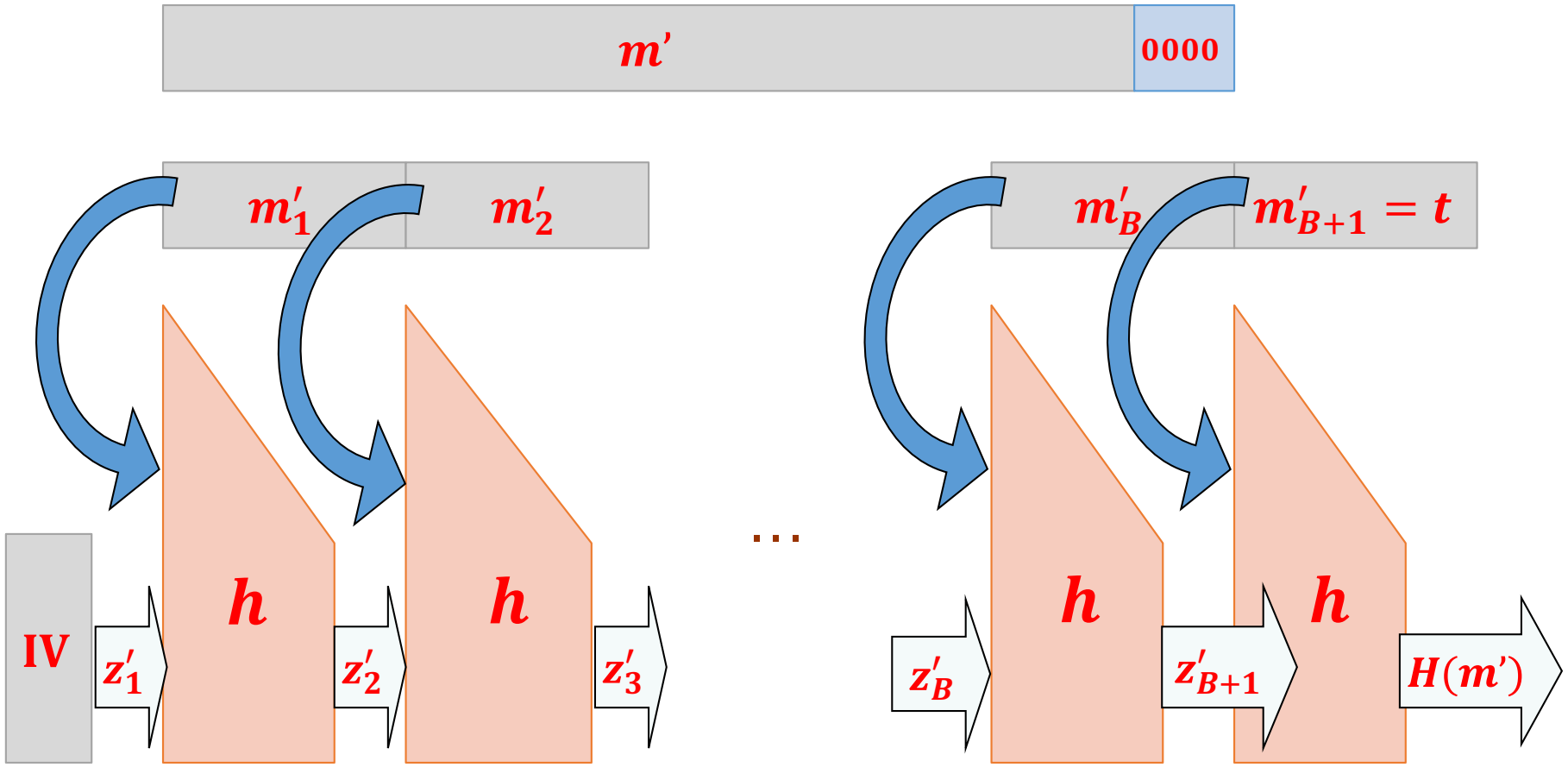
$|m| = |m'|$

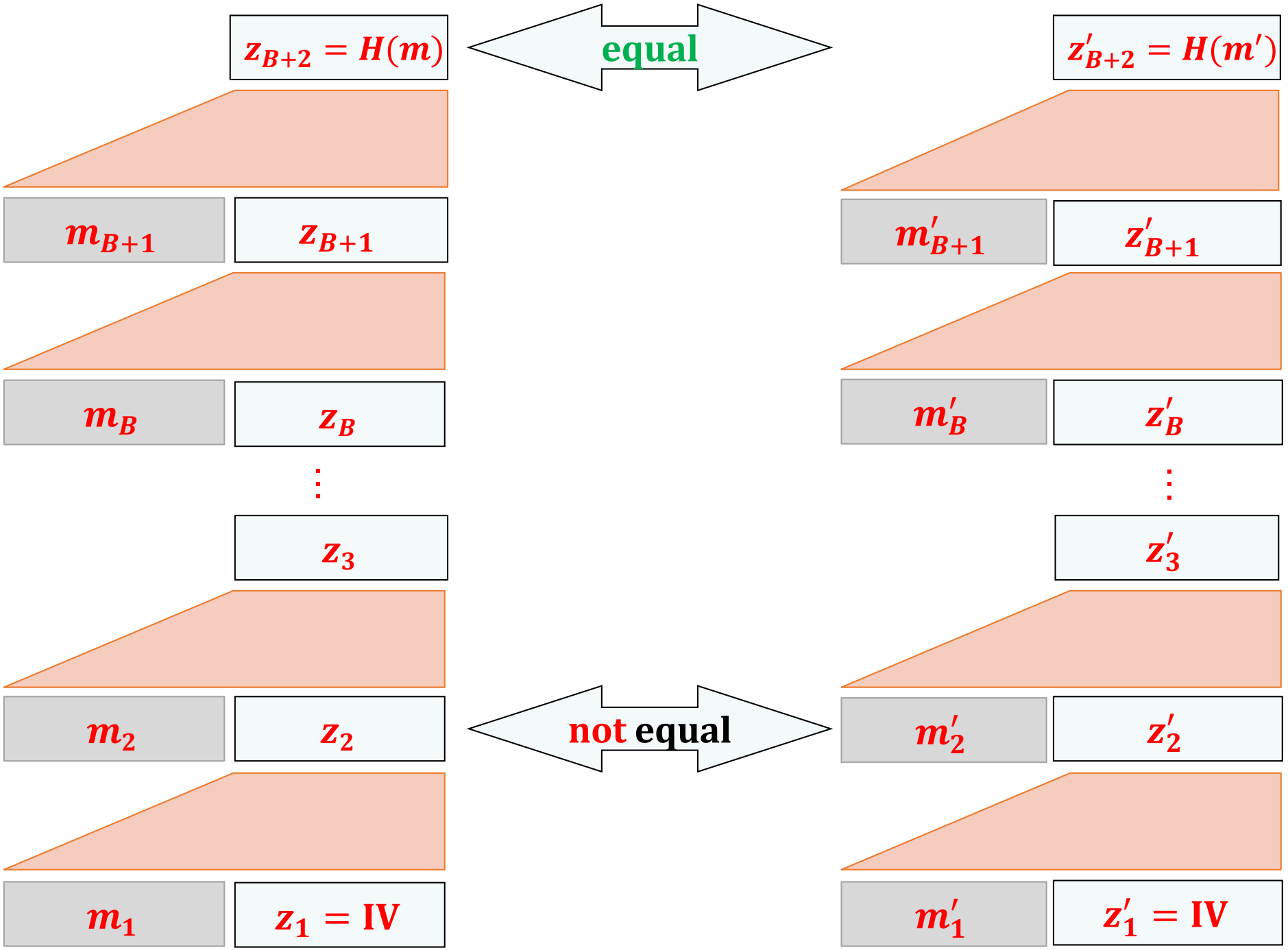
Some notation:

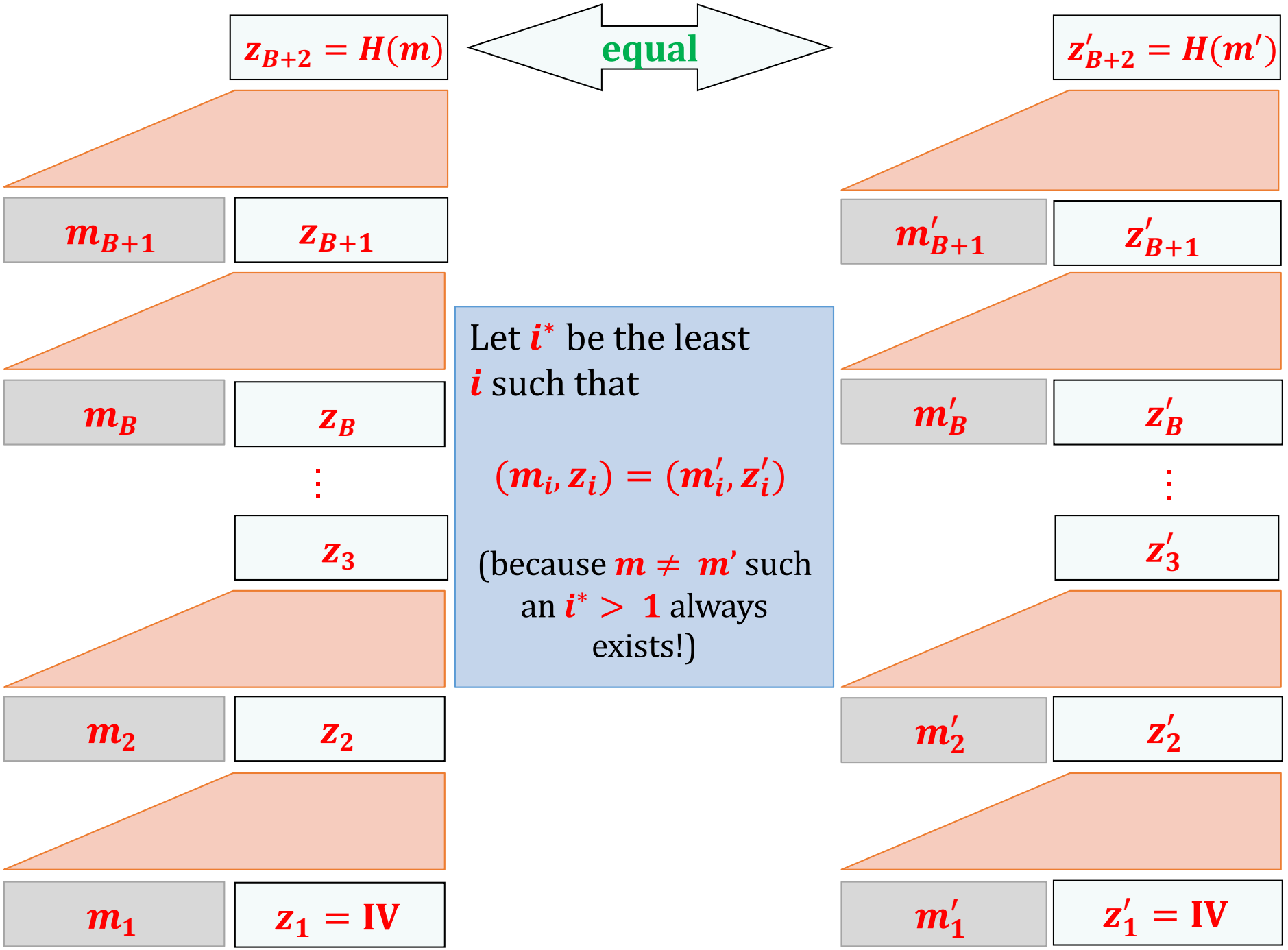


$|m| = |m'|$

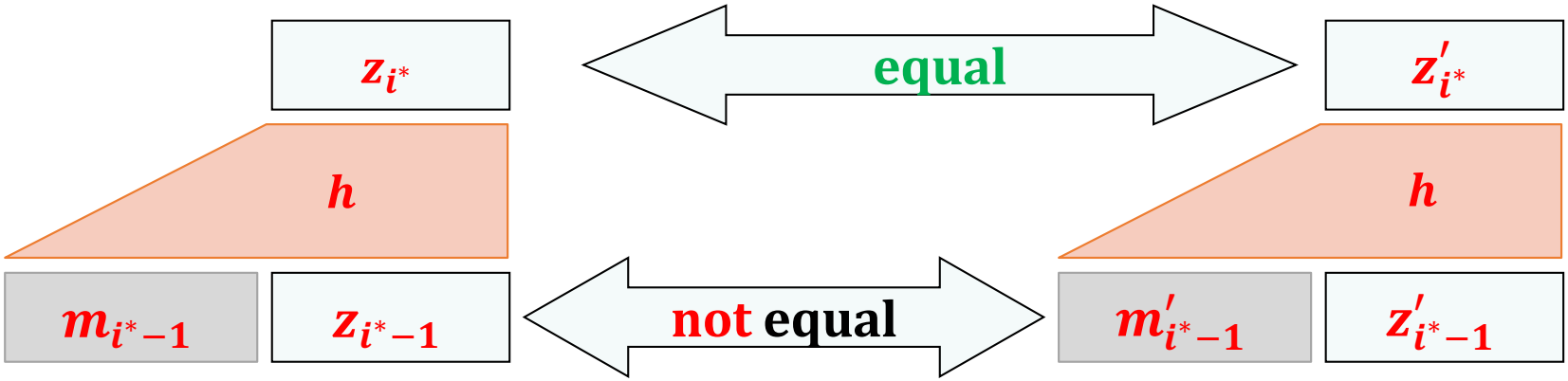
For m':



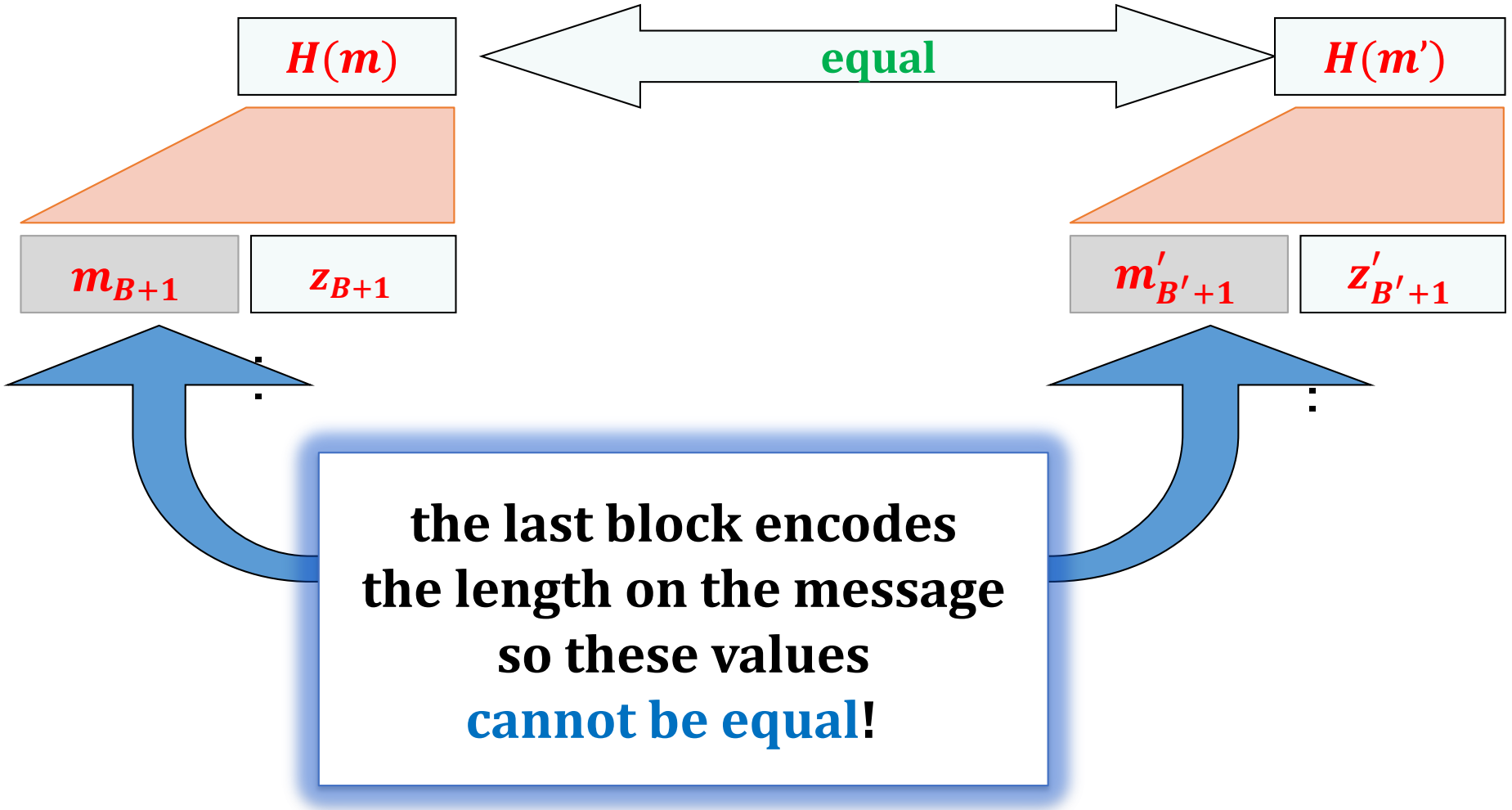

$z_{B+2} = H(m)$
equal
$z'_{B+2} = H(m')$
m_{B+1}
z_{B+1}
m'_{B+1}
z'_{B+1}
m_B
z_B
m'_B
z'_B
z_3
z'_3
m_2
z_2
not equal
m'_2
z'_2
m_1
$z_1 = \mathrm{IV}$
m'_1
$z'_1 = \mathrm{IV}$


$z_{B+2} = H(m)$
equal
$z'_{B+2} = H(m')$
m_{B+1}
z_{B+1}
m'_{B+1}
z'_{B+1}
m_B
z_B
m'_B
z'_B
z_3
z'_3
m_2
z_2
m'_2
z'_2
m_1
$z_1 = \mathrm{IV}$
m'_1
$z'_1 = \mathrm{IV}$
Let i^* be the least i such that
$(m_i, z_i) = (m'_i, z'_i)$
(because $m \neq m'$ such an $i^* > 1$ always exists!)

So, we have found a collision!



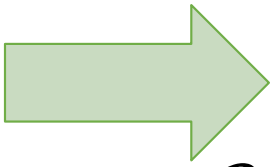
Case 2: $|m| \neq |m'|$



So, again we have found a collision!

Plan

1. Introduction and definitions
2. Hash function design paradigms
    1. Merkle-Damgård transform
    2. Sponge construction
3. Other uses of hash functions
4. Real-life constructions



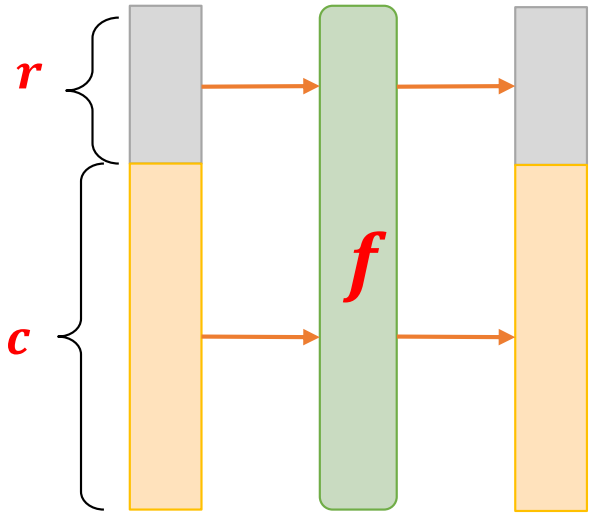
Sponge construction

main parameters:

c – "capacity"

r – "rate"

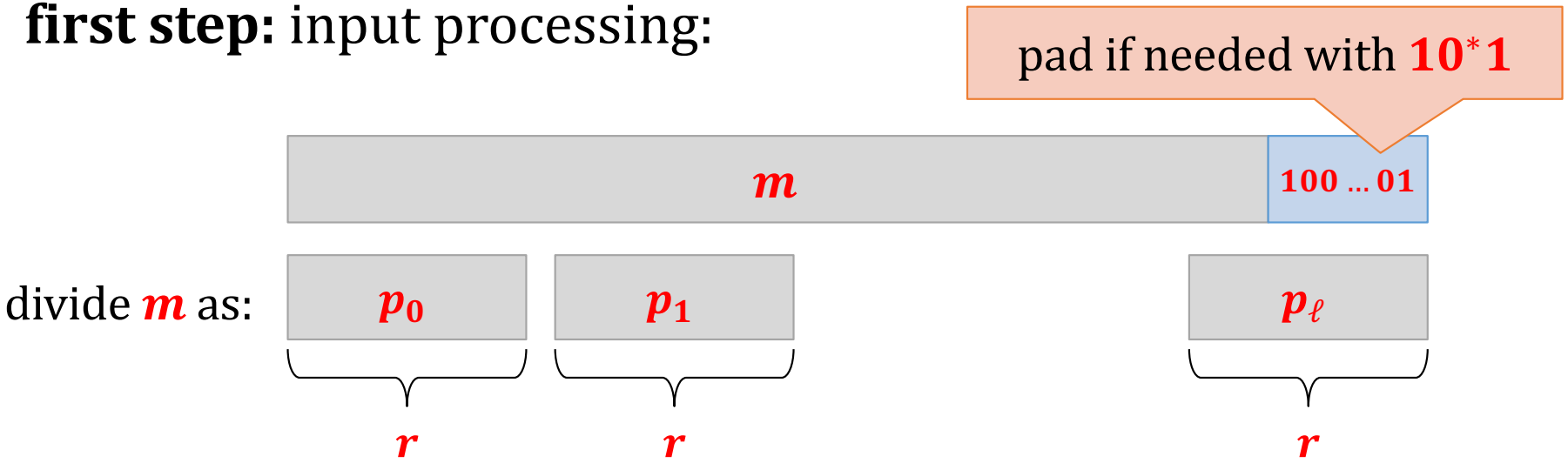
$b := c + r$ – "state width"



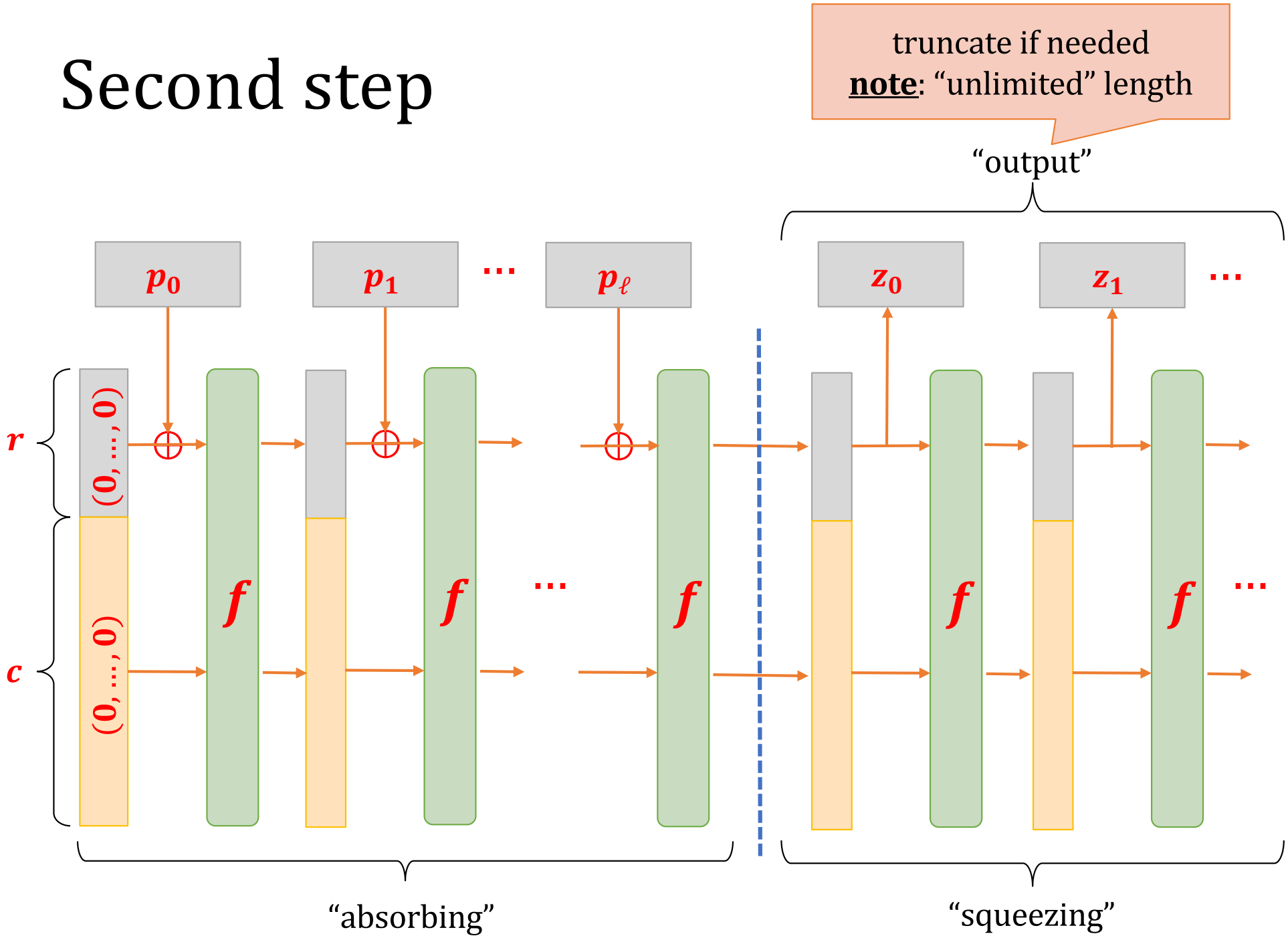
main ingredient:

a function $f: \{0,1\}^{r+c} \to \{0,1\}^{r+c}$

first step: input processing:



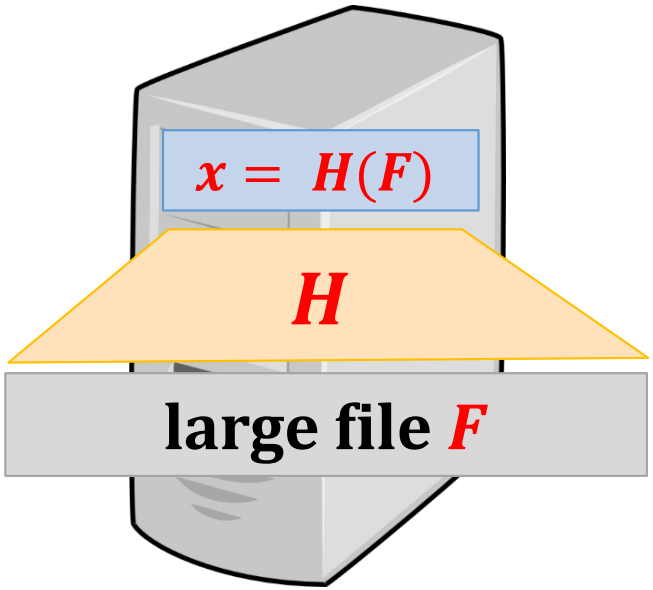
Second step


truncate if needed
note: "unlimited" length
"output"
p_0
p_1
p_ℓ
z_0
z_1
r
c
$(0, \ldots, 0)$
$(0, \ldots, 0)$
f
"absorbing"
"squeezing"

Plan

1. Introduction and definitions
2. Hash function design paradigms
3. Other uses of hash functions
    1. Merkle trees
    2. Practical randomness extraction and the random oracle model
    3. Password storage and Proofs of Work
4. Real-life constructions

Consider again file fingerprinting

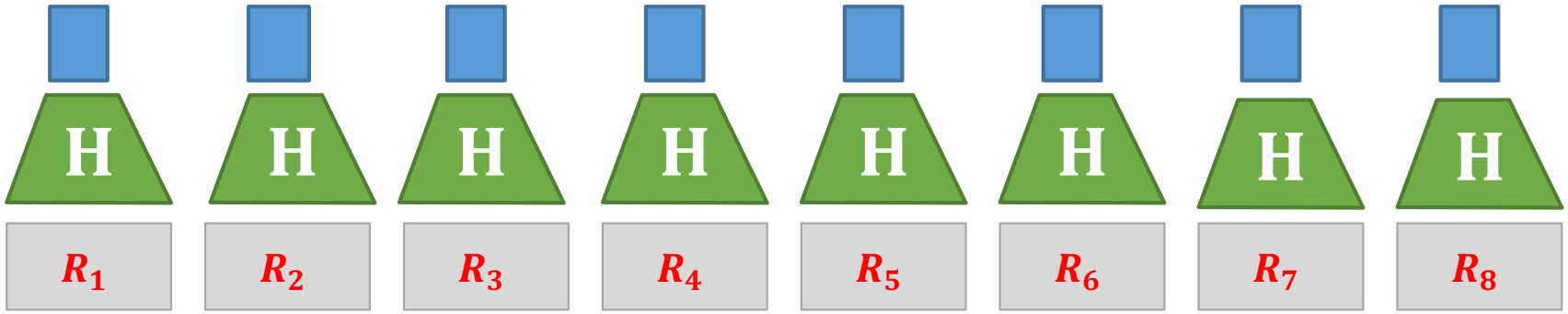


Fingerprinting large files

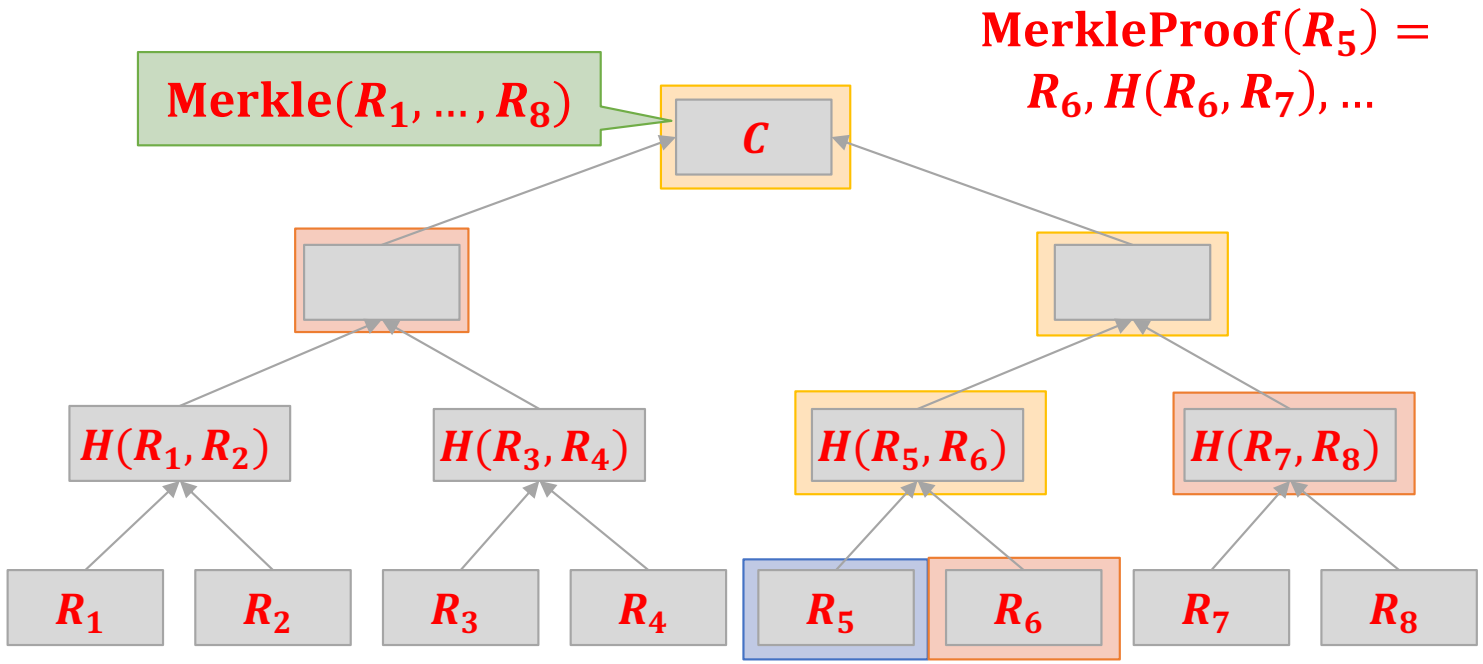
Suppose a file F consists of many smaller blocks $R_1, \ldots, R_n$, and the user may want to **access only one of them**.

Question: How to "fingerprint F"?

Naive solution: fingerprint each of them independently.



Better solution: construct a **Merkle tree:**



Merkle trees allow to efficiently prove that each block R_i was included into the hash C.

This is done by sending $\mathbf{MerkleProof}(R_i)$.

Easy to see: if H is collision resistant then so is **Merkle**.

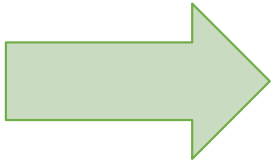
File sharing

BitTorrent, **Gnutella**, **Gnutella2**, and **Direct Connect P2P**... - a variant of the idea from the previous slides:

- files in the peer-to-peer networks are **identified by their hashes**
- each file consists of "**pieces**"
- the users download the pieces from each other.
- some of them use **Merkle trees.**

Plan

1. Introduction and definitions
2. Hash function design paradigms
3. Other uses of hash functions
    1. Merkle trees
    2. Practical randomness extraction and the random oracle model
    3. Password storage and Proofs of Work
4. Real-life constructions



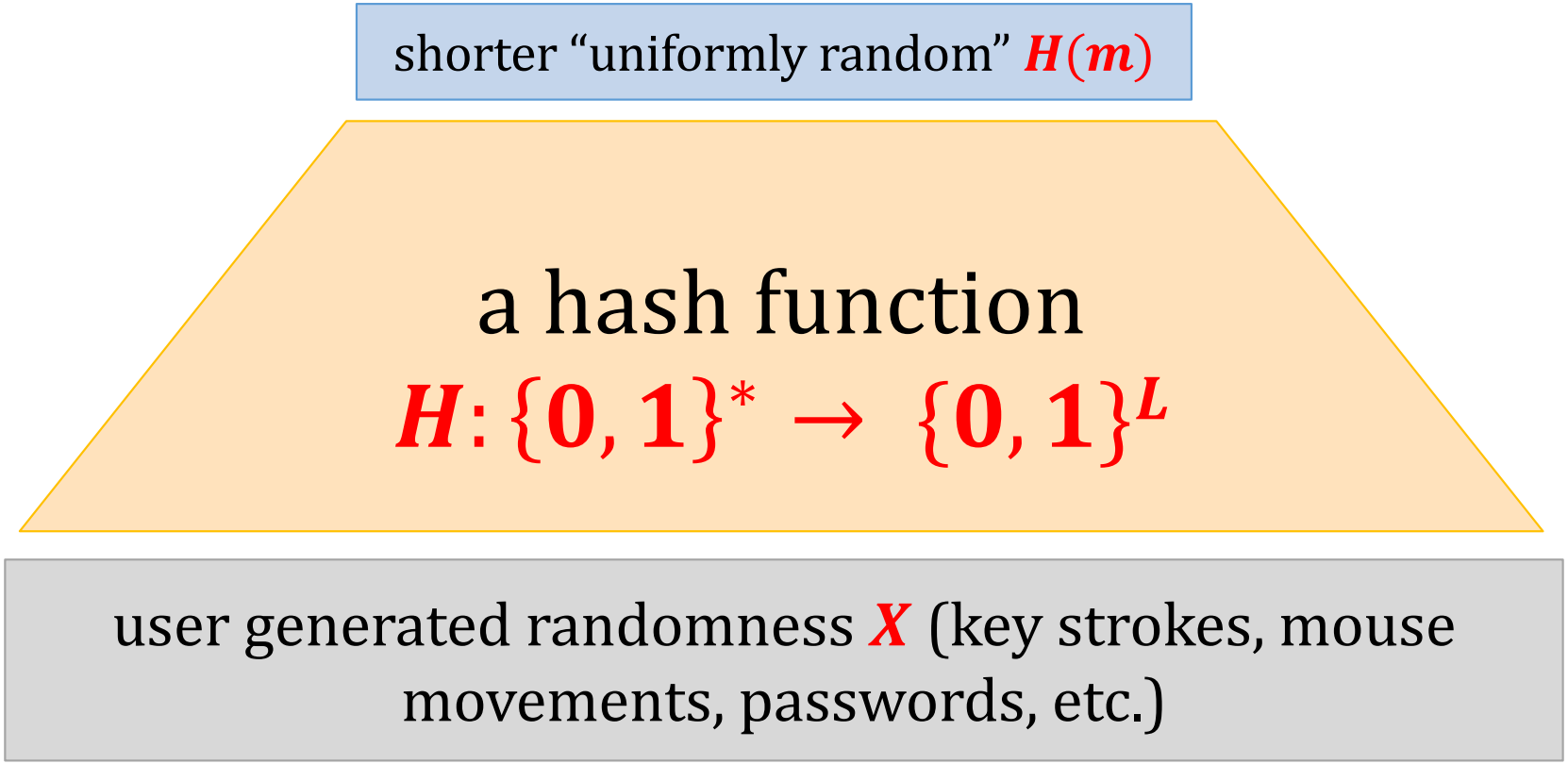
How the outputs of hash functions look in real life?

- SHA3 “**Wydział Matematyki, Informatyki i Mechaniki**”
  016372eff8b4c4068ebd56dc8fcb59374c5dc7b27107a24903339fa28301d586
- SHA3 “**Wydział Matematyki, Informtyki i Mechaniki**”
  b2094ca714e7da9546d4b31c0a3c6ae8180b239f244a9eb7f5085989fc27806f
- SHA3 “**Wydział Matematyki Informatyki i Mechaniki**”
  1ee4ab789fbee41cd17c839f01ebb2227d56ebfe79cbfbe3984055d1184369b6

Observation: the outputs on different inputs are “unrelated” and “completely random”.

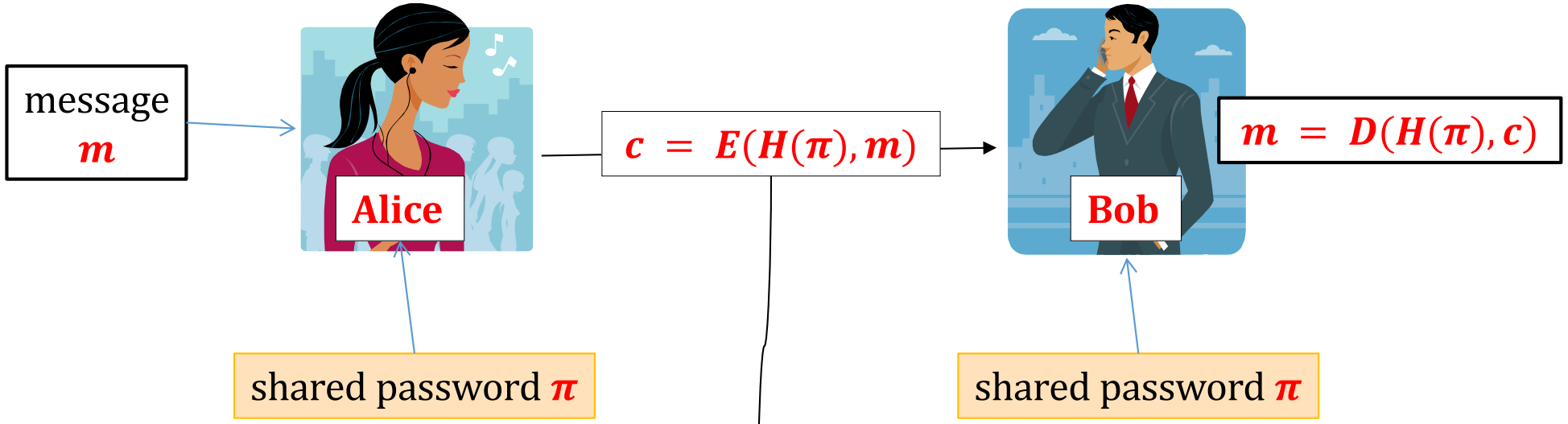
we will formalize this property in a moment

Example of how this property is used: deriving “uniformly random keys” from “non-uniform randomness"



Example: password-based encryption

H – hash function
(E, D) – encryption scheme



Warning:
there exist much better solutions for this problem



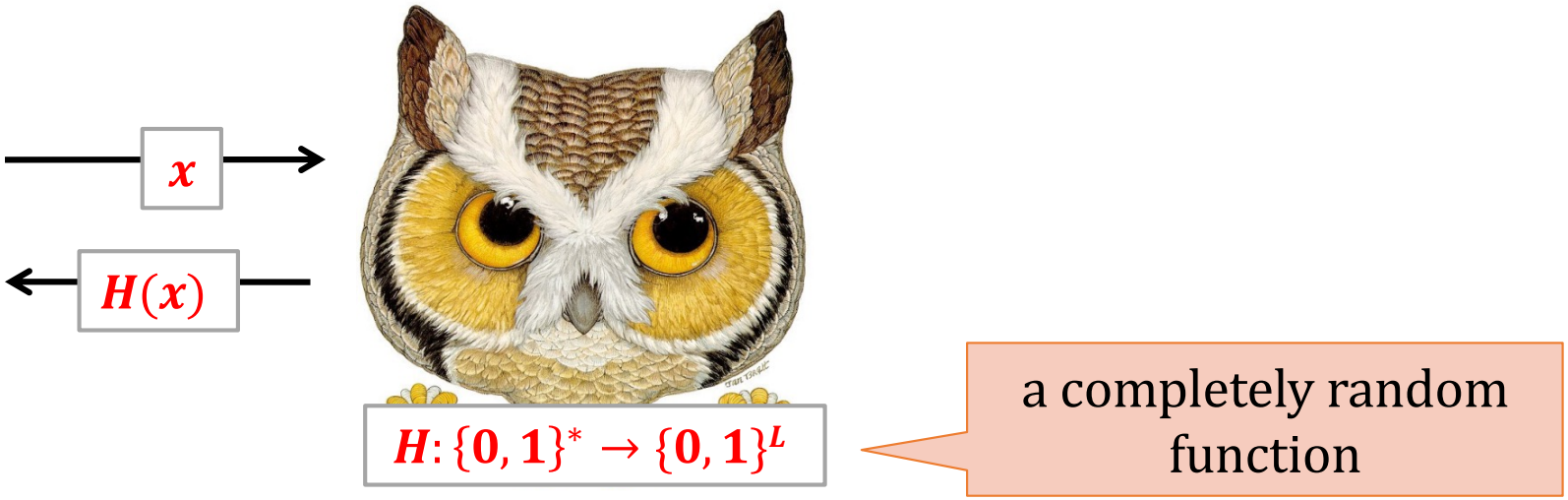
Informally:
The only thing that Eve can do is to examine all possible passwords .

Random oracle model

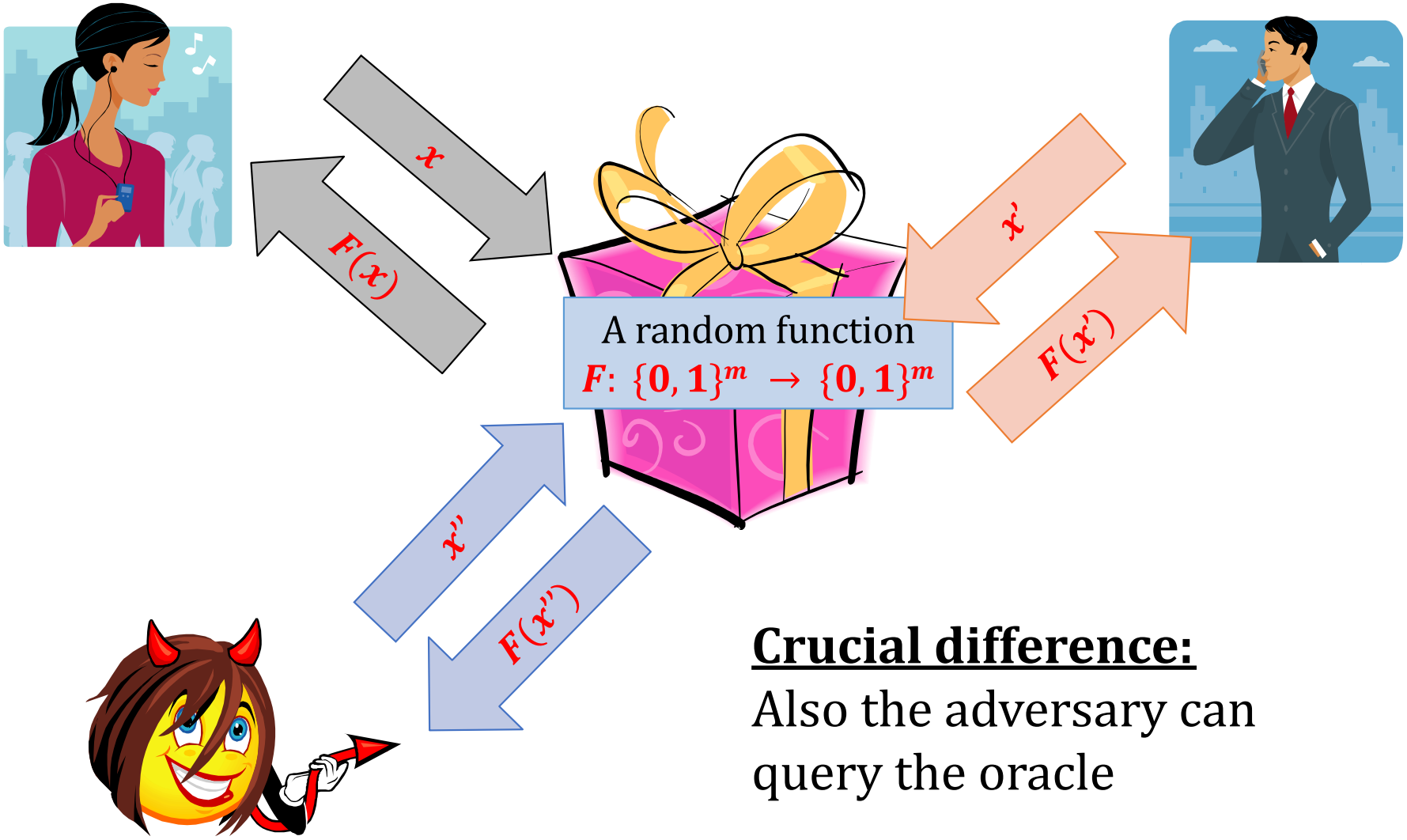
[Fiat, Shamir: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. 1986]

[Bellare, Rogaway: Random Oracles are Practical: A Paradigm for Designing Efficient Protocols, 1993]

Idea: model the hash function as a **random oracle**.



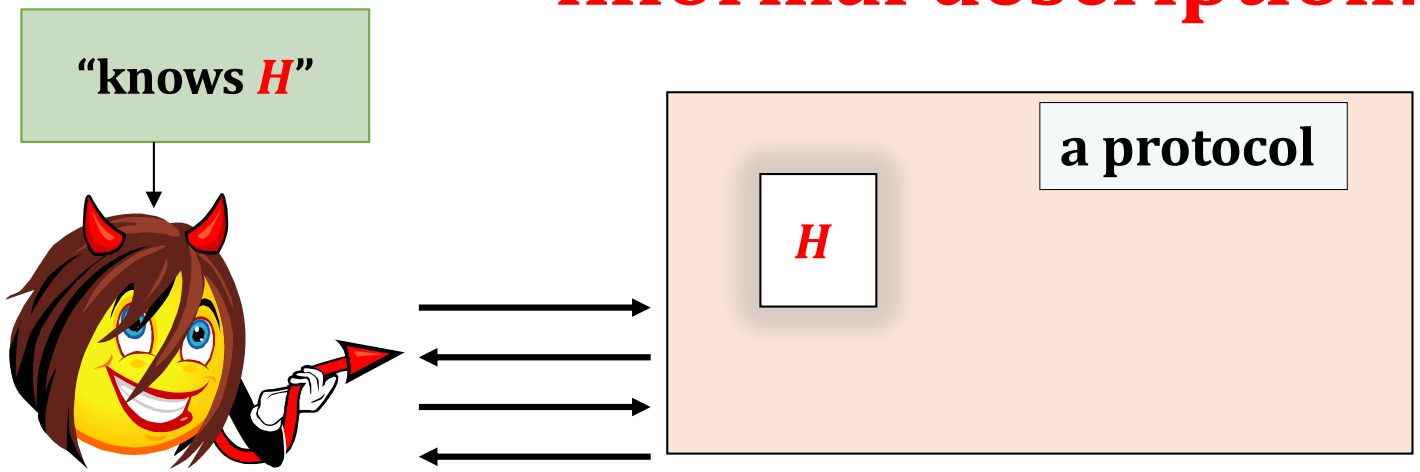
Remember the pseudorandom functions?



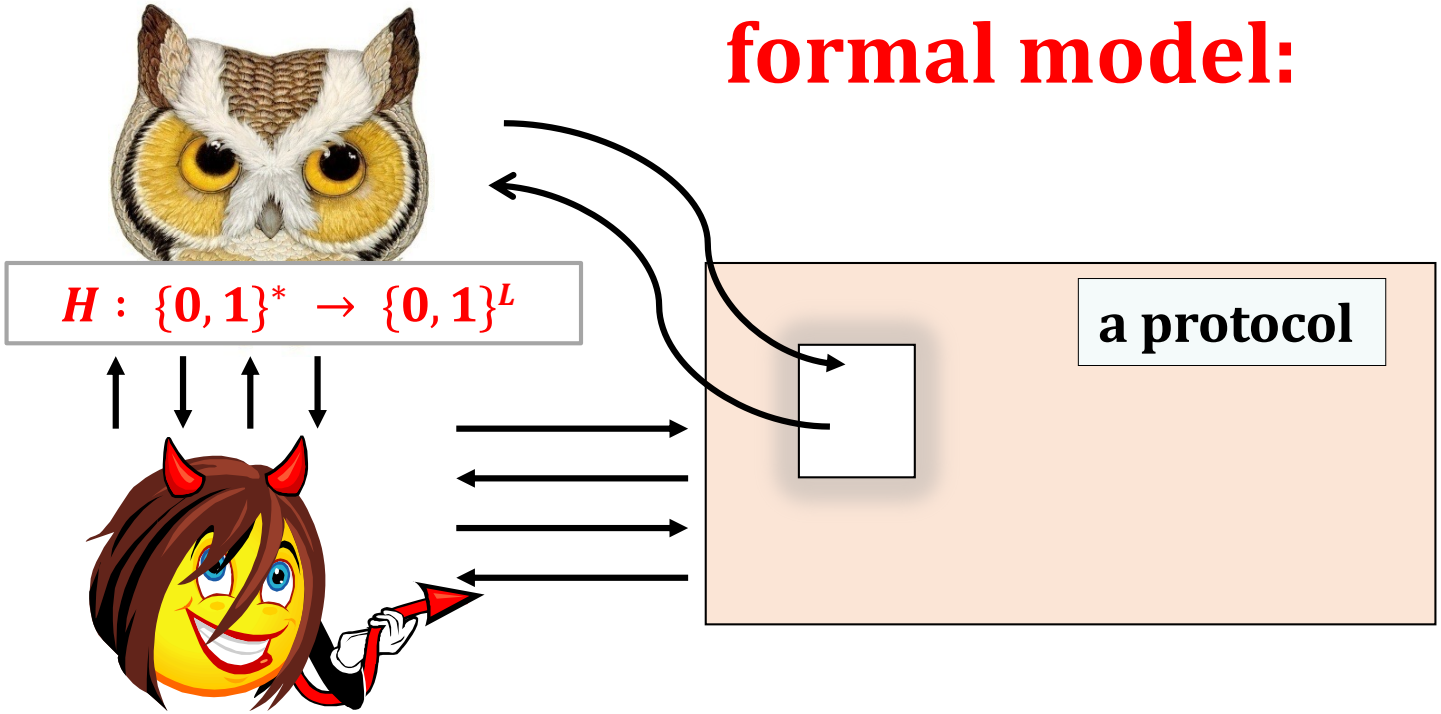
<u>Crucial difference:</u>

Also the adversary can query the oracle

informal description:



formal model:



Every call to H is replaced with a query to the oracle.

also the adversary is allowed to query the oracle.

How would we use it in the proof?

shorter "uniformly random" $H(X)$

a hash function

$H: \{0,1\}^* \rightarrow \{0,1\}^L$

user generated randomness X



As long as the adversary never queried the oracle on X the value $H(X)$ "looks completely random to him".

Criticism of the Random Oracle Model

[Canetti, Goldreich, Halevi: **The random oracle methodology, revisited**. 1998]

There exists a signature scheme that is

- **secure** in **ROM**

but

- is **not secure** if the random oracle is replaced with **any** real hash function.

This example is **very artificial**. No "realistic" example of this type is know.

Terminology

Model without the random oracles:

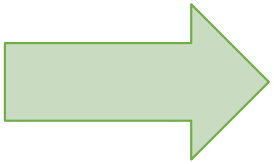
- "**plain model**"
- "**cryptographic model**"

Random Oracle Model is also called:
the "**Random Oracle Heuristic**".

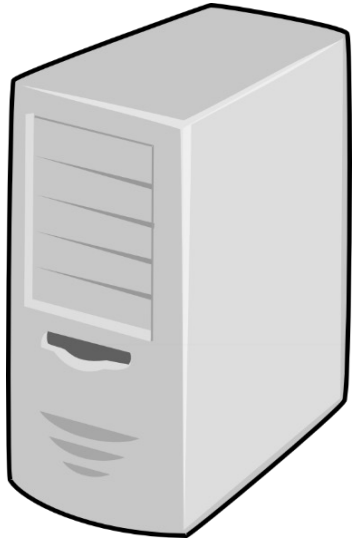
Common view: a proof in **ROM** is better than nothing.

Plan

1. Introduction and definitions
2. Hash function design paradigms
3. Other uses of hash functions
    1. Merkle trees
    2. Practical randomness extraction and the random oracle model
    3. Password storage and Proofs of Work
4. Real-life constructions



Password storage



Simple idea: instead of storing user's passwords π in plaintext store their hashes.

Better: "salted hashes" $(s, H(s, \pi))$

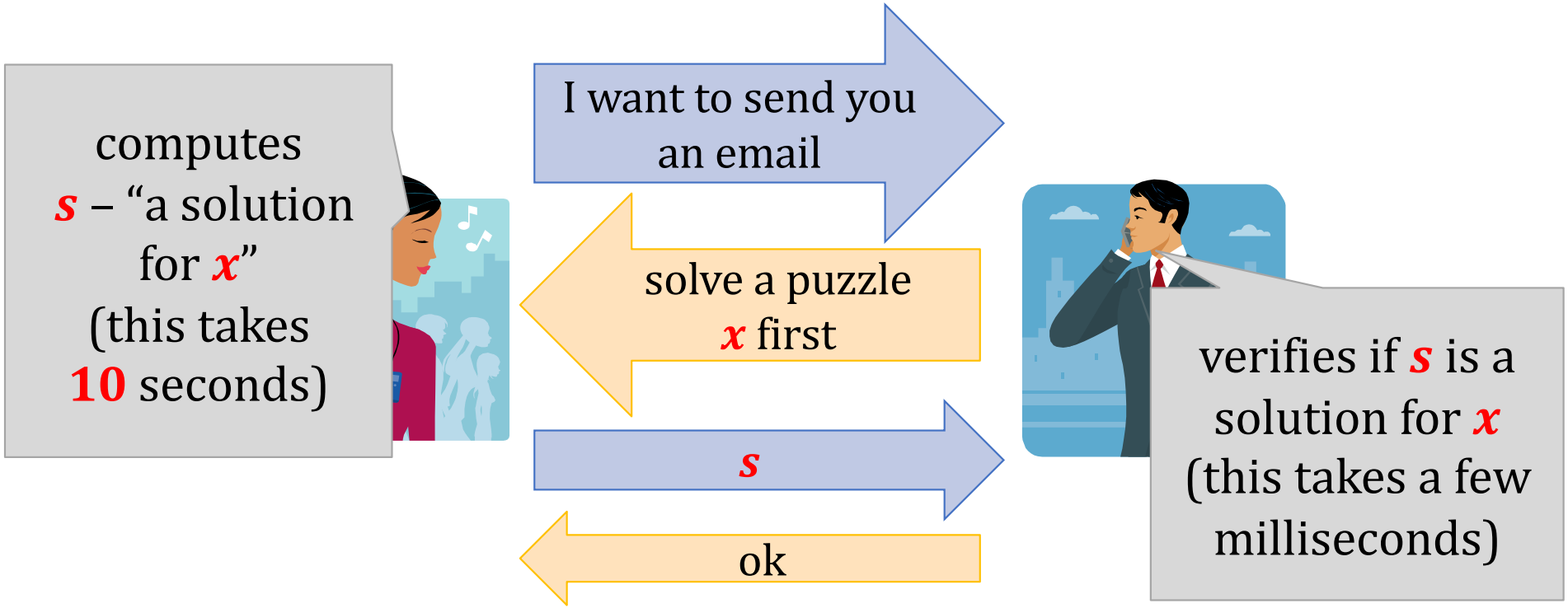
advantages:

1. makes "precomputation attacks" harder [**exercise**]
2. if two users have the same password then the stored values are different.

Proofs of work

Introduced by **Dwork and Naor** [Crypto 1992] as a countermeasure against spam.

Basic idea: Force users to do some computational work: solve a **moderately difficult** "puzzle" (checking correctness of the solution has to be fast).



A simple hash-based PoW

H – a hash function whose computation takes time **TIME(H)**



random **x**



Prover

finds s such that

$H(s, x)$ starts with n zeros (in binary)

s

Verifier

checks if

$H(s, x)$ starts with **n** zeros

salt | "hardness parameter

(in **ROM**) takes expected time $2^n \cdot$ **TIME(H)**

takes time **TIME(H)**

This **PoW** is used in Bitcoin.

Problem

Computing typical hash functions is much faster when done in **parallel** and in **hardware** (this can give advantage to a powerful adversary).

For example "**Bitcoin mining**" is done almost entirely on ASICs

AntMiner S7



Advertised Capacity:
4.73 Th/s

Power Efficiency:
0.25 W/Gh

Weight:
8.8 pounds

Guide:
Yes

Price:
$479.95

Avalon6



Advertised Capacity:
3.5 Th/s

Power Efficiency:
0.29 W/Gh

Weight:
9.5 pounds

Guide:
No

Price:
$499.95

SP20 Jackson



Advertised Capacity:
1.3-1.7 Th/s

Power Efficiency:
0.65 W/Gh

Weight:
20 pounds

Guide:
Yes

Price:
$248.99

Idea for a solution

Design hash functions whose computation **needs to lot of memory**, so it's hard to implement it efficiently in hardware

Example: **scrypt** hash function introduced in:

> **Colin Percival, *Stronger Key Derivation via Sequential Memory-Hard Functions*, 2009**.

Used in **Litecoin** 

Has one practical drawback: it's access pattern is **data-dependent** (hence: it reveals the input).

bad for the side-channel resilience

Password Hashing Competition

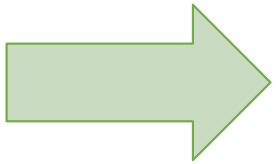
- announced in **2013**
- run by an **independent panel** of experts
- **website**: password-hashing.net
- winner (2015): **Argon2** (by Biryukov, Dinu, and Khovratovic)

broken (together with several other competition finalists) by Alwen, Gaži, Kamath, Klein, Osang, Pietrzak, Reyzin, Rolínek, Rybár: **On the Memory-Hardness of Data-Independent Password-Hashing Functions**, Aug 2016

Several improvements to **Argon2** (e.g.: "**Argon2i 1.3**") broken

Plan

1. Introduction and definitions
2. Hash function design paradigms
3. Other uses of hash functions
4. Real-life constructions



Fact

There exists a generic attack on any hash function

$$H: \{0,1\}^* \to \{0,1\}^n$$

that finds a **collision with probability** $\frac{1}{2}$ and works in **time and space**

$$O\left(2^{\frac{n}{2}}\right).$$

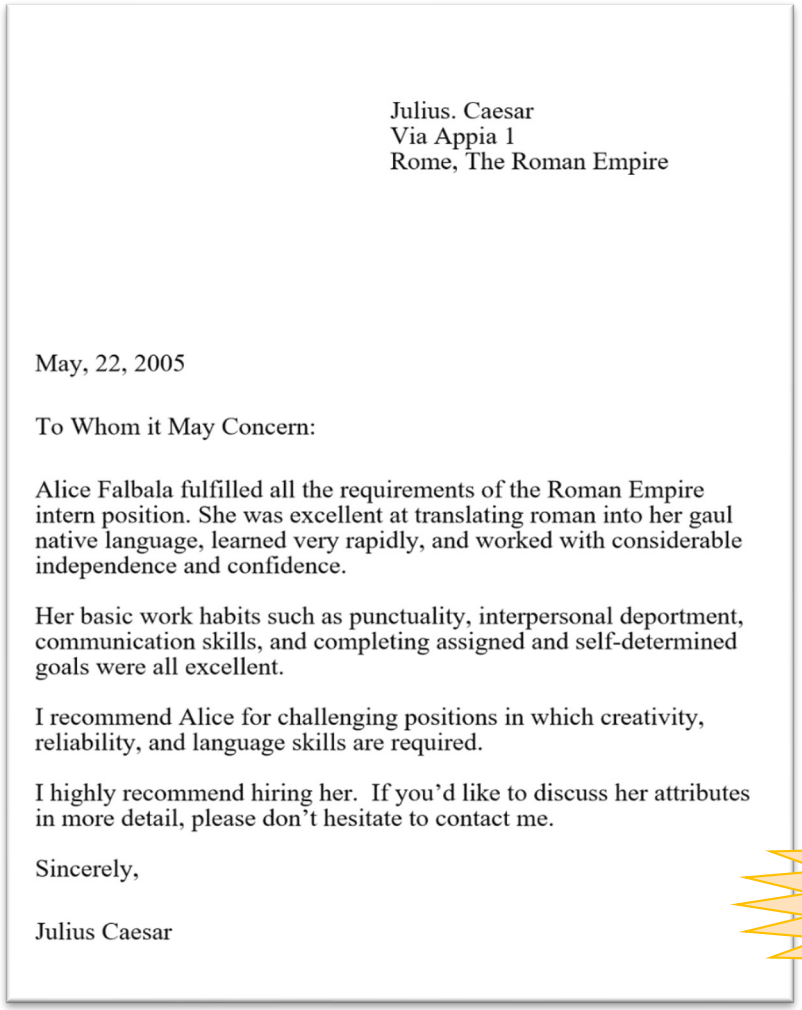
It's called a **birthday attack** and we will discuss it during the exercises.

Consequence: to achieve "m bits of security" one needs to set $n = 2 \cdot m$.

MD5 (Message-Digest Algorithm 5)

- based on the **Merkle-Damgard paradigm**
- **output length**: **128 bits**,
- **designed** by **Rivest** in **1991**,
- in **1996**, **Dobbertin** found collisions in the compressing function of **MD5**,
- in **2004** a group of **Chinese mathematicians** designed a method for finding collisions in **MD5**,

June 2005: researchers at the Bochum University produce 2 postscript documents with the same **MD5** hash



Julius. Caesar
Via Appia 1
Rome, The Roman Empire

May, 22, 2005

To Whom it May Concern:

Alice Falbala fulfilled all the requirements of the Roman Empire intern position. She was excellent at translating roman into her gaul native language, learned very rapidly, and worked with considerable independence and confidence.

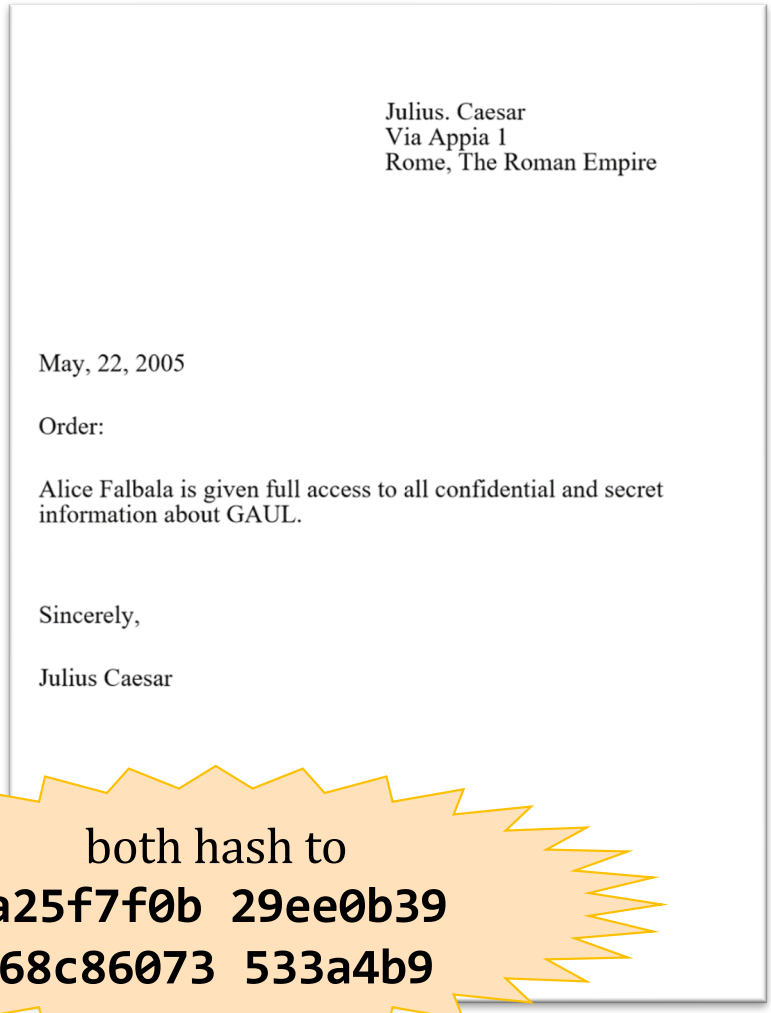
Her basic work habits such as punctuality, interpersonal deportment, communication skills, and completing assigned and self-determined goals were all excellent.

I recommend Alice for challenging positions in which creativity, reliability, and language skills are required.

I highly recommend hiring her. If you'd like to discuss her attributes in more detail, please don't hesitate to contact me.

Sincerely,

Julius Caesar



Julius. Caesar
Via Appia 1
Rome, The Roman Empire

May, 22, 2005

Order:

Alice Falbala is given full access to all confidential and secret information about GAUL.

Sincerely,

Julius Caesar

both hash to
a25f7f0b 29ee0b39
68c86073 533a4b9

This is done by exploiting the redundancy in postscript.

Colliding certificates

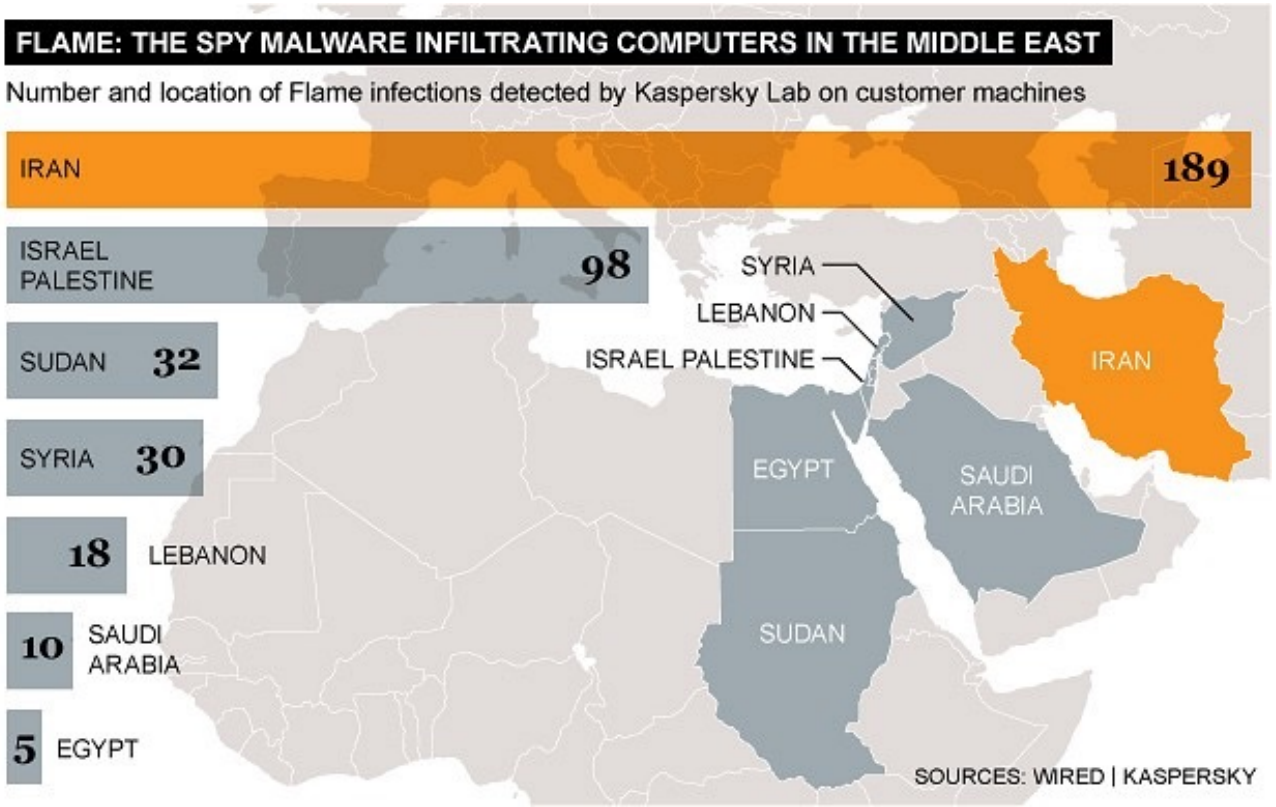
2005 and 2006: A. Lenstra, X. Wang, and B. de Weger found **X.509** certificates with different public keys and the same **MD5** hash.

(we will discuss the **X.509** certificates later)

Two certificates for **different names** ("Arjen K. Lenstra" and "Marc Stevens") **and different public keys**.

Flame malware

attack on the Microsoft Windows Update Mechanism exploiting **MD5** collision



SHA-1 (Secure Hash Algorithm)

- based on the **Merkle-Damgard paradigm**
- **output length**: **160 bits**,
- designed in **1993** by the **NSA**,
- in **2005 Xiaoyun Wang, Andrew Yao and Frances Yao** presented an attack that runs in time 2^{63}.
- Oct 2015: **[Stevens, Karpman, and Peyrin: Freestart collision on full SHA-1]**
  a collision in the compression function in 2^{57} **SHA-1** evaluations.
- Feb 2017: **[Stevens, Bursztein, Karpman, Albertini , Markov]**
  a collision in full **SHA-1**
- Improved in **[Leurent and Peyrin 2020]**

The collision that was found in 2015

	Input 1
IV1	50 6b 01 78 ff 6d 18 90 20 22 91 fd 3a de 38 71 b2 c6 65 ea
M1	9d 44 38 28 a5 ea 3d f0 86 ea a0 fa 77 83 a7 36 33 24 48 4d af 70 2a aa a3 da b6 79 d8 a6 9e 2d 54 38 20 ed a7 ff fb 52 d3 ff 49 3f c3 ff 55 1e fb ff d9 7f 55 fe ee f2 08 5a f3 12 08 86 88 a9
SHA1_compression_function (IV1,M1)	f0 20 48 6f 07 1b f1 10 53 54 7a 86 f4 a7 15 3b 3c 95 0f 4b

	Input 2
IV2	50 6b 01 78 ff 6d 18 91 a0 22 91 fd 3a de 38 71 b2 c6 65 ea
M2	3f 44 38 38 81 ea 3d ec a0 ea a0 ee 51 83 a7 2c 33 24 48 5d ab 70 2a b6 6f da b6 6d d4 a6 9e 2f 94 38 20 fd 13 ff fb 4e ef ff 49 3b 7f ff 55 04 db ff d9 6f 71 fe ee ee e4 5a f3 06 04 86 88 ab
SHA1_compression_function (IV2,M2)	f0 20 48 6f 07 1b f1 10 53 54 7a 86 f4 a7 15 3b 3c 95 0f 4b

Hardware used in **[Stevens, Karpman, and Peyrin: Freestart collision on full SHA-1]**:

*"We have computed the SHA-1 freestart collision on **Kraken**, our 64-GPU cluster. More precisely Kraken is composed of 16 nodes, each node being made of simple, cheap and widely available hardware: 4 GTX-970 GPUs, 1 Haswell i5-4460 processor and 16GB of RAM."*



Reaction of the industry

Microsoft may block SHA1 certificates sooner than expected

Encrypted sites running old certificates will be inaccessible from modern browsers.

By Zack Whittaker for Zero Day | November 9, 2015 -- 13:16 GMT (13:16 GMT) | Topic: Security

Mozilla Security Blog

OCT 20 2015

Continuing to Phase Out SHA-1 Certificates

Richard Barnes

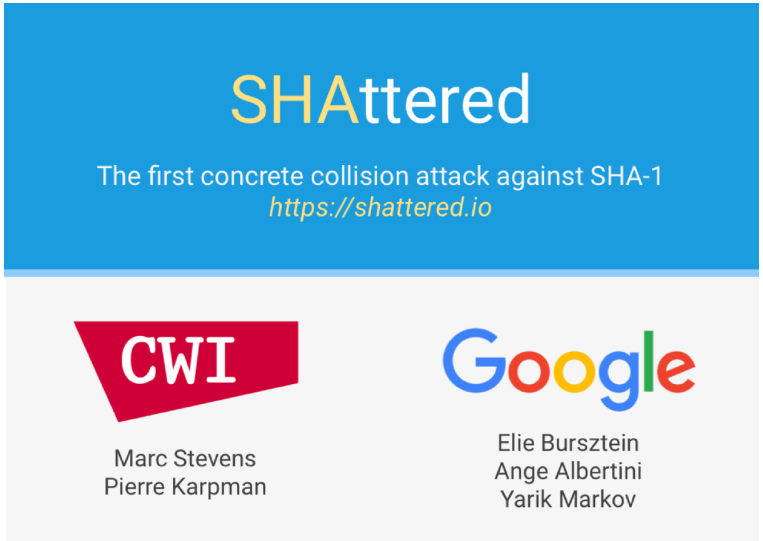
In our previous blog post about phasing out certificates with SHA-1 based signature algorithms, we said that we planned to take a few actions with regard to SHA-1 certificates:

1. Add a security warning to the Web Console to remind developers that they should not be using a SHA-1 based certificates
2. Show the "Untrusted Connection" error whenever a SHA-1 certificate issued after January 1, 2016, is encountered in Firefox
3. Show the "Untrusted Connection" error whenever a SHA-1 certificate is encountered in Firefox after January 1, 2017

The attack from 2017



Two colliding pdf files:





An unexpected victim of this attack

NEWS

SHA-1 collision can break SVN code repositories

The WebKit repository was corrupted after someone committed two colliding PDF files to it



By Lucian Constantin

Romania Correspondent, IDG News Service

FEB 27, 2017 10:23 AM PT

A new hash algorithm: SHA-3

Selected by the **National Institute of Standards and Technology (NIST)** in an open competition.

5 finalists: **BLAKE, Grøstl, JH, Keccak, Skein**.

Winner (2012): **Keccak**.

Keccak is the new standard

- **Designers**: Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche.
- **NIST standard** from 2015 (selected in an open competition).
- Now called **SHA-3 (Secure Hash Algorithm 3)**.
- Based on the **sponge construction**. Example of recommended parameters:

Instance	Output size d	Rate r = block size	Capacity c
SHA3-224	224	1152	448
SHA3-256	256	1088	512
SHA3-384	384	832	768
SHA3-512	512	576	1024

©2026 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation*.